

KOHAKURAG: A SIMPLE RAG FRAMEWORK WITH HIERARCHICAL DOCUMENT INDEXING

Shih-Ying Yeh^{1,2,3,†} Yueh-Feng Ku¹ Ko-Wei Huang¹ Buu-Khang Tu¹

¹National Tsing Hua University ²Comfy Org Research ³Kohaku-Lab

[†]Corresponding author: kohaku@kblueleaf.net • Code: <https://github.com/KohakuBlueleaf/KohakuRAG>

ABSTRACT

Retrieval-augmented generation (RAG) systems that answer questions from document collections face compounding difficulties when high-precision citations are required: flat chunking strategies sacrifice document structure, single-query formulations miss relevant passages through vocabulary mismatch, and single-pass inference produces stochastic answers that vary in both content and citation selection. We present KohakuRAG, a hierarchical RAG framework that preserves document structure through a four-level tree representation (document → section → paragraph → sentence) with bottom-up embedding aggregation, improves retrieval coverage through an LLM-powered query planner with cross-query reranking, and stabilizes answers through ensemble inference with abstention-aware voting. We evaluate on the WattBot 2025 Challenge, a benchmark requiring systems to answer technical questions from 32 documents with $\pm 0.1\%$ numeric tolerance and exact source attribution. KohakuRAG achieves first place on both public and private leaderboards (final score 0.861), as the only team to maintain the top position across both evaluation partitions. Ablation studies reveal that prompt ordering (+80% relative), retry mechanisms (+69%), and ensemble voting with blank filtering (+1.2pp) each contribute substantially, while hierarchical dense retrieval alone matches hybrid sparse-dense approaches (BM25 adds only +3.1pp). We release KohakuRAG as open-source software at <https://github.com/KohakuBlueleaf/KohakuRAG>.

1 INTRODUCTION

Large Language Models (LLMs) demonstrate remarkable capabilities across natural language processing tasks (Brown et al., 2020; OpenAI, 2025a), yet suffer from hallucination, outdated knowledge, and inability to access domain-specific information absent from training data (Huang et al., 2023). Retrieval-Augmented Generation (RAG) addresses these limitations by grounding LLM outputs in external knowledge sources (Lewis et al., 2020; Gao et al., 2024).

The WattBot 2025 Challenge (Endemann et al., 2025) presents a compelling testbed for RAG systems: given 32 reference documents ($\sim 500\text{K}$ tokens) on AI energy consumption, systems must answer approximately 300 technical questions with $\pm 0.1\%$ numeric tolerance, cite exact source documents, and explicitly abstain when evidence is insufficient. This combination of scale, precision, and citation requirements exposes fundamental limitations of standard RAG approaches.

Standard RAG systems (Lewis et al., 2020; Huang and Huang, 2024) employ flat chunking that partitions documents into fixed-length segments, destroying structural boundaries and complicating precise citation tracking (Günther et al., 2024). Retrieval is typically performed with a single query formulation, which misses relevant passages when user questions use different vocabulary than source documents, e.g., querying “PUE” when documents discuss “power usage effectiveness” (Wang et al., 2023a; Jagerman et al., 2023). Finally, a single LLM inference produces stochastic answers that vary across runs in both content and citation selection, with models often abstaining unnecessarily when evidence is present but difficult to locate.

We present KohakuRAG, a hierarchical RAG framework that addresses each of these challenges through a matched mechanism (Figure 1):

- **Hierarchical document indexing.** To preserve document structure, we represent documents as four-level trees (document → section → paragraph → sentence) and compute embeddings bottom-up

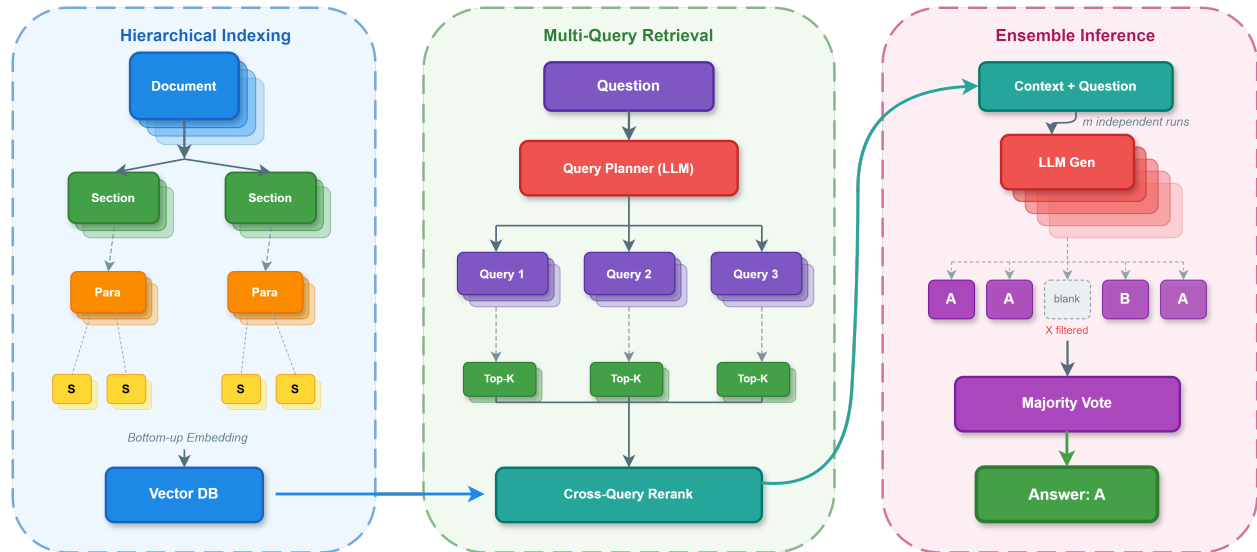


Figure 1: Overview of KohakuRAG. **Left (Hierarchical Indexing):** Documents are parsed into tree structures with sections, paragraphs (Para), and sentences (S). Sentence embeddings are computed and aggregated bottom-up to parent levels, then stored in a Vector DB. **Center (Multi-Query Retrieval):** Given a question, the Query Planner (LLM) generates multiple related queries, each retrieving Top-K results that are merged via Cross-Query Reranking. **Right (Ensemble Inference):** Context and question are sent to the LLM for m independent runs; blank responses are filtered (X), and majority voting produces the final answer.

via length-weighted aggregation. This provides natural citation boundaries at each granularity level while capturing compositional semantics.

- **Multi-query retrieval with cross-query reranking.** To bridge the vocabulary gap between questions and source documents, an LLM-powered query planner expands questions into multiple retrieval queries covering alternative phrasings and terminology. Cross-query reranking then leverages consensus signals, where nodes retrieved by more queries rank higher, to surface the most relevant passages.
- **Ensemble inference with abstention-aware voting.** To mitigate answer instability, we aggregate m independent inference runs via majority voting. By filtering out blank responses before aggregation, we prevent conservative runs from dominating when evidence exists but is hard to locate.

Our ablation studies quantify the impact of each design choice: prompt reordering (placing context before the question) yields +80% relative improvement; the retry mechanism provides +69% at low retrieval depth; and ensemble voting with blank filtering adds +1.2pp at $n=9$. Hierarchical dense retrieval alone achieves competitive performance, with BM25 augmentation adding only +3.1pp, suggesting that keyword matching provides diminishing returns when retrieval structure is sufficiently rich. KohakuRAG achieves first place on both the public and private leaderboards of the WattBot 2025 Challenge with a final score of 0.861. We were the only team to maintain the top position across both evaluation partitions, demonstrating that our ensemble-based approach generalizes reliably to unseen data.

Our main contributions are:

- A hierarchical document indexing scheme that preserves structural relationships through four-level tree representations with bottom-up embedding aggregation, enabling precise citation tracking.
- An LLM-powered query planner with cross-query reranking that improves retrieval coverage by expanding questions into multiple formulations and leveraging consensus signals.
- An ensemble inference mechanism with abstention-aware voting that aggregates multiple runs while filtering unnecessary abstentions, the dominant error mode (26.8% of failures).
- Comprehensive experiments showing that hierarchical dense retrieval alone achieves competitive performance on citation-heavy tasks, and that prompt ordering and retry mechanisms contribute more than hybrid retrieval strategies.

The remainder of this paper is organized as follows. Section 2 provides background on text retrieval and RAG systems. Section 3 discusses related work. Section 4 details our hierarchical indexing approach, retrieval strategies, and answering pipeline. Section 5 describes our experimental setup, and Section 6 presents results and analysis. Section 7 concludes with limitations and future directions.

2 PRELIMINARIES

We review the key research areas underlying our approach and formalize the WattBot 2025 Challenge.

2.1 LARGE LANGUAGE MODELS

Large Language Models (LLMs) based on the transformer architecture (Vaswani et al., 2017) have progressed from BERT (Devlin et al., 2019) and GPT-3 (Brown et al., 2020) to frontier models including GPT-5 (OpenAI, 2025a), Gemini 3 (Google DeepMind, 2025), Claude Opus 4.5 (Anthropic, 2025), and Grok 4.1 (xAI, 2025). Open-weight alternatives have also reached frontier quality: Mistral 3 (Mistral AI, 2025) with 675B total parameters and 41B active, Gemma 3 (Gemma Team et al., 2025), and GPT-oss-120B (OpenAI, 2025b). Despite strong reasoning capabilities, LLMs suffer from hallucination (Huang et al., 2023), outdated knowledge, and inability to access domain-specific information, motivating retrieval-augmented approaches. Vision-language models (VLMs) extend LLMs with visual understanding: Qwen3-VL (Bai et al., 2025) achieves state-of-the-art on MMMU and video understanding with native 256K-token multimodal context, while earlier work including LLaVA (Liu et al., 2023) and Qwen-VL (Bai et al., 2023; Wang et al., 2024b) established key architectural patterns. Recent surveys (Yin et al., 2024) document rapid MLLM progress enabling document understanding where figures and tables contain critical information.

2.2 TEXT EMBEDDINGS AND RETRIEVAL

Dense retrieval represents queries and documents as vectors in $\mathbb{R}^d$, computing relevance as $\text{sim}(q, d) = E(q)^\top E(d)$ for encoder E . DPR (Karpukhin et al., 2020) demonstrated that contrastive learning outperforms BM25 (Robertson and Zaragoza, 2009), with improvements from Contriever (Izacard et al., 2022) and ANCE (Xiong et al., 2021). Modern embeddings achieve state-of-the-art on MTEB (Muennighoff et al., 2023) and its multilingual expansion MMTEB (Enevoldsen et al., 2025). Notable models include Sentence-BERT (Reimers and Gurevych, 2019), GTE (Li et al., 2023), E5 (Wang et al., 2024a), BGE (Xiao et al., 2024b), and the Jina series: Jina v3 (Sturua et al., 2024) introduces task-specific LoRA adapters, while Jina v4 (Günther et al., 2025) unifies text and image embeddings through a 3.8B parameter multimodal architecture with Matryoshka representations (Kusupati et al., 2022) supporting dimensions from 128 to 2048.

Dense Vector Search. Efficient retrieval over large embedding collections requires approximate nearest neighbor (ANN) algorithms. HNSW (Malkov and Yashunin, 2018) constructs hierarchical navigable small world graphs enabling logarithmic search complexity with high recall. The Faiss library (Johnson et al., 2021; Douze et al., 2025) provides optimized implementations of multiple indexing strategies including inverted file indices (IVF), product quantization (PQ), and HNSW, supporting billion-scale search on both CPU and GPU. These algorithms trade exact retrieval for dramatic speedups, typically achieving >95% recall while reducing latency by orders of magnitude compared to brute-force search.

Sparse Vector Search. Sparse methods represent documents as high-dimensional vectors with most entries zero, enabling efficient inverted index lookup. BM25 (Robertson and Zaragoza, 2009) remains a strong baseline, computing term-frequency statistics without training. Learned sparse methods like SPLADE (Formal et al., 2021) use neural networks to learn term weights and expand documents with semantically related terms, combining the efficiency of sparse representations with learned relevance signals. Hybrid approaches that combine dense and sparse retrieval consistently outperform either method alone (Mandikal et al., 2024), as lexical and semantic signals capture complementary relevance aspects.

Two-stage pipelines use fast first-stage retrieval followed by expensive reranking. Cross-encoders (Nogueira and Cho, 2019) jointly encode query-document pairs with $O(n)$ cost; BGE-M3 (Chen et al., 2024) provides multi-granularity reranking. Late interaction models like ColBERT (Khattab and Zaharia, 2020; Santhanam et al., 2022) compute token-level MaxSim: $\text{score}(q, d) = \sum_i \max_j \mathbf{q}_i^\top \mathbf{d}_j$, balancing efficiency and accuracy. ColPali (Faysse et al., 2025) extends this to multimodal document retrieval over images without OCR.

2.3 RETRIEVAL-AUGMENTED GENERATION

RAG (Lewis et al., 2020) conditions generation on retrieved documents: given corpus $\mathcal{D}$ and query q , retrieve $\mathcal{D}_q = \text{top-}k(\{d : \text{sim}(q, d)\})$, then generate $p(a|q, \mathcal{D}_q)$. Comprehensive surveys (Gao et al., 2025; Singh et al., 2025; Gao et al., 2024; Zhao et al., 2024; Huang and Huang, 2024) categorize systems as Naïve, Advanced (with query rewriting and reranking), Modular RAG, or Agentic RAG with autonomous retrieval decisions. Query expansion addresses vocabulary mismatch: Query2Doc (Wang et al., 2023a) generates pseudo-documents, while other approaches use rewriting (Ma et al., 2023) or direct expansion (Jagerman et al., 2023). Document segmentation impacts quality; fixed chunking loses context, while late chunking (Günther et al., 2024) encodes documents before segmentation. Our approach preserves natural structure as a tree $T = (V, E)$ with bottom-up embedding aggregation. For multimodal documents, M3DocRAG (Cho et al., 2024) shows combined text-visual retrieval improves performance. Self-consistency (Wang et al., 2023b) and ensemble methods (Chen et al., 2025) improve robustness via majority voting.

2.4 PROBLEM FORMULATION: THE WATTBOT 2025 CHALLENGE

Document-grounded question answering spans several benchmarks with varying characteristics. Natural Questions (Kwiatkowski et al., 2019) pairs real user queries with Wikipedia articles, requiring systems to extract short answers from long documents. TriviaQA (Joshi et al., 2017) provides trivia questions with multiple evidence documents, testing multi-hop reasoning. PDFTriage (Saad-Falcon et al., 2023) focuses on structured document QA with complex layouts. These benchmarks primarily evaluate answer extraction; citation accuracy and abstention behavior receive less attention.

The WattBot 2025 Challenge (Endemann et al., 2025) extends document-grounded QA to AI energy estimation, requiring systems to answer technical questions about power consumption, water usage, carbon footprint, and performance characteristics. We formalize the task as follows.

Task Definition. Given a document corpus $\mathcal{D} = \{d_1, \dots, d_{32}\}$ and a question q , the system must produce a tuple (a, R, b) where:

- $a \in \mathbb{R} \cup \mathcal{V}$ is the answer, either numeric or categorical from vocabulary $\mathcal{V}$
- $R \subseteq \{1, \dots, 32\}$ is the set of cited document identifiers
- $b \in \{0, 1\}$ indicates abstention when evidence is insufficient

The corpus comprises 32 reference documents (research papers, technical reports, presentation slides) totaling approximately 500K tokens, exceeding typical LLM context limits and necessitating retrieval-based approaches.

Evaluation Protocol. The challenge imposes stringent requirements across three dimensions:

Answer Accuracy (75% weight): Numeric answers must fall within $\pm 0.1\%$ relative tolerance:

$$\text{ValueScore}(a, a^*) = \mathbf{1} \left[\left| \frac{a - a^*}{a^*} \right| \leq 0.001 \right] \quad (1)$$

Categorical answers require exact string match after normalization.

Citation Precision (15% weight): Measured via Jaccard similarity between predicted and ground-truth reference sets:

$$\text{RefScore}(R, R^*) = \frac{|R \cap R^*|}{|R \cup R^*|} \quad (2)$$

Hallucination Avoidance (10% weight): Systems must explicitly abstain ($b = 1$) rather than hallucinate when evidence is insufficient. Incorrect answers when abstention was appropriate incur penalties.

The final score combines these components:

$$\text{Score} = 0.75 \times \text{ValueScore} + 0.15 \times \text{RefScore} + 0.10 \times \text{HallucinationScore} \quad (3)$$

requiring systems to jointly optimize retrieval coverage, answer extraction, and uncertainty quantification.

3 RELATED WORK

We discuss prior work on document chunking strategies, query expansion methods, and ensemble approaches for RAG systems.

Document Chunking and Hierarchical Indexing. Standard RAG systems partition documents into fixed-length chunks for embedding and retrieval (Lewis et al., 2020; Gao et al., 2024). While simple to implement, fixed-length chunking destroys semantic boundaries and complicates citation tracking. Late chunking (Günther et al., 2024) addresses boundary artifacts by encoding documents before segmentation, preserving contextual information across chunk boundaries. However, late chunking still produces flat representations without explicit hierarchical structure. Our approach differs by constructing explicit tree representations that preserve document organization (sections, paragraphs, sentences) and enable precise provenance tracking through hierarchical node identifiers.

Query Expansion and Multi-Query Retrieval. Single queries often lack sufficient context for optimal retrieval, motivating query expansion techniques. Query2Doc (Wang et al., 2023a) generates pseudo-documents to enrich query representations. Other approaches employ query rewriting (Ma et al., 2023) or direct expansion through prompting (Jagerman et al., 2023). We adopt LLM-powered query planning that generates multiple semantically related queries, combined with cross-query reranking that leverages consensus signals, where nodes retrieved by multiple queries rank higher, providing implicit voting across different query formulations.

Self-Reflection and Corrective RAG. Recent work explores mechanisms for RAG systems to assess and improve their own outputs. Self-RAG (Asai et al., 2023) trains models to generate special reflection tokens that indicate when to retrieve, whether retrieved passages are relevant, and whether generated content is supported by evidence. CRAG (Yan et al., 2024) introduces a corrective retrieval mechanism that evaluates retrieval quality and triggers web search when initial retrieval is insufficient. Our retry mechanism shares similar motivation: when the model indicates insufficient evidence (abstention), we expand context and retry. However, our approach operates at inference time without requiring specialized training.

Ensemble Methods for RAG. Self-consistency (Wang et al., 2023b) improves reasoning by sampling multiple outputs and selecting the most common answer through majority voting. Recent surveys on LLM ensembles (Chen et al., 2025) document various aggregation strategies for combining predictions across models or runs. Our ensemble mechanism extends these ideas to RAG with reference-aware voting modes that jointly consider answer values and citation sets, with explicit handling of abstention cases where some runs indicate insufficient evidence.

Multimodal Document Understanding. Technical documents frequently contain critical information in figures and tables. M3DocRAG (Cho et al., 2024) demonstrates that combining text and visual retrieval improves performance on multi-page document understanding. ColPali (Faysse et al., 2025) enables document retrieval directly from page images using late interaction over vision embeddings. Our framework supports both caption-based retrieval (where vision models generate textual descriptions during indexing) and direct image embedding through Jina v4 (Günther et al., 2025), enabling flexible multimodal retrieval strategies.

4 METHODOLOGY

We present KohakuRAG, a hierarchical RAG framework designed for document-grounded question answering with precise citation requirements. Our approach addresses three key challenges: (1) preserving document structure during indexing, (2) improving retrieval coverage through multi-query planning, and (3) achieving robust answers via ensemble inference. Algorithm 1 presents the complete pipeline.

4.1 HIERARCHICAL DOCUMENT INDEXING

Standard RAG systems employ flat chunking strategies that partition documents into fixed-length segments. While simple, this approach destroys document structure and complicates citation tracking. We instead preserve the natural hierarchy of technical documents through tree-structured indexing.

Document Parsing. Given a document d , we construct a tree $T_d = (V_d, E_d)$ with four levels of granularity. The root node represents the entire document. Section nodes correspond to logical divisions (chapters, numbered sections). Paragraph nodes capture coherent text blocks within sections. Sentence nodes form the leaves, containing individual sentences extracted via rule-based segmentation. Each node v stores its text content t_v , a unique identifier following the pattern `doc:seci:pj:sk`, and metadata including source document ID and position indices.

The KohakuRAG pipeline operates in three phases. First, given question q , an LLM-powered query planner generates n semantically related queries $\mathcal{Q} = \{q_1, \dots, q_n\}$ that cover different phrasings and aspects of the information need. Each query retrieves top- k nodes via dense similarity search, and results are aggregated through cross-query reranking that prioritizes nodes retrieved by multiple queries. Second, retrieved nodes are expanded with hierarchical context, including parent paragraphs and sibling sentences, to provide the LLM with sufficient surrounding information. Third, m independent inference runs generate answer candidates $(a^{(j)}, R^{(j)}, b^{(j)})$ where $b^{(j)}$ indicates abstention. If $b^{(j)} = 1$ and retries remain, we increase k and re-retrieve. Finally, ensemble voting with blank filtering produces the final answer.

Algorithm 1 KohakuRAG Pipeline

Require: q , index $\mathcal{I}$, k , n_{queries} , m_{runs}
Ensure: (a, R, b)

```

1: function KOHAKURAG( $q, \mathcal{I}, k, n, m$ )
2:    $\mathcal{Q} \leftarrow \text{PLAN\_QUERIES}(q, n)$ 
3:    $\mathcal{R} \leftarrow \emptyset$ 
4:   for  $q_i \in \mathcal{Q}$  do
5:      $\mathcal{R} \leftarrow \mathcal{R} \cup \text{RETRIEVE}(\mathcal{I}, q_i, k)$ 
6:   end for
7:    $\mathcal{R} \leftarrow \text{RERANK}(\mathcal{R})$ 
8:    $\mathcal{C} \leftarrow \text{EXPAND\_CONTEXT}(\mathcal{R})$ 
9:    $\mathcal{A} \leftarrow \emptyset$ 
10:  for  $j = 1$  to  $m$  do
11:     $(a^{(j)}, R^{(j)}, b^{(j)}) \leftarrow \text{GENERATE}(\mathcal{C}, q)$ 
12:    if  $b^{(j)} = 1$  and retries left then
13:       $k \leftarrow k + \Delta k$ ; goto line 4
14:    end if
15:     $\mathcal{A} \leftarrow \mathcal{A} \cup \{(a^{(j)}, R^{(j)}, b^{(j)})\}$ 
16:  end for
17:  return  $\text{ENSEMBLE\_VOTE}(\mathcal{A})$ 
18: end function

```

For documents containing figures and tables, we treat each visual element as a special paragraph node with `attachment.type=image`. We employ a Vision Language Model (VLM) from the Qwen-VL family (Bai et al., 2023; 2025) to generate captions, which are stored as the node’s text content. This enables text-based retrieval of visual information while preserving the hierarchical structure.

Bottom-Up Embedding Propagation. We compute embeddings in a bottom-up manner to capture compositional semantics. Let $\mathcal{C}(v)$ denote the children of node v . For leaf nodes (sentences), we directly compute embeddings using a text encoder E :

$$\mathbf{e}_v = E(t_v), \quad \text{if } \mathcal{C}(v) = \emptyset \quad (4)$$

For internal nodes, the embedding is computed as a weighted average of child embeddings:

$$\mathbf{e}_v = \frac{\sum_{c \in \mathcal{C}(v)} w_c \cdot \mathbf{e}_c}{\sum_{c \in \mathcal{C}(v)} w_c} \quad (5)$$

where $w_c = |t_c|$ is the token count of child c . This length-weighted averaging ensures that longer, more informative children contribute proportionally more to the parent embedding. The resulting embeddings at paragraph and section levels capture the aggregate semantics of their constituents.

Embedding Models. We support two embedding backends. Jina Embeddings v3 (Sturua et al., 2024) provides 768-dimensional text embeddings with 8K context length and task-specific LoRA adapters, trained with multi-stage contrastive learning similar to GTE (Li et al., 2023), E5 (Wang et al., 2024a), and BGE (Xiao et al., 2024b). These models achieve state-of-the-art results on the MTEB benchmark (Muennighoff et al., 2023). Jina Embeddings v4 (Günther et al., 2025) extends this with multimodal capability and Matryoshka representations (Kusupati et al., 2022) (128–2048 dimensions), enabling direct image embedding for cross-modal retrieval.

Vector Storage. Embeddings are stored using `sqlite-vec`, a lightweight SQLite extension for vector similarity search. Our middleware layer, KohakuVault, provides a unified interface for key-value metadata storage and vector indexing within a single database file. This design enables efficient deployment without external database dependencies.

4.2 MULTI-QUERY RETRIEVAL

User questions often lack sufficient context for optimal retrieval. A query like “What is the PUE of Google’s data centers?” may miss relevant passages discussing “power usage effectiveness” or “energy efficiency metrics.” We address this through LLM-powered query planning.

Query Planner. Given question q , an LLM generates n semantically related queries $\{q_1, \dots, q_n\}$ that cover different phrasings and aspects of the information need. The planner is prompted to produce queries that:

- Rephrase the original question using alternative terminology
- Expand abbreviations and technical terms
- Decompose compound questions into sub-queries
- Add contextual keywords likely to appear in relevant passages

Dense Retrieval. For each planned query q_i , we embed it and retrieve the top- k matching nodes via cosine similarity search:

$$\mathcal{R}_i = \text{top-}k(\{v \in V : \cos(\mathbf{e}_{q_i}, \mathbf{e}_v)\}, k) \quad (6)$$

Retrieval is restricted to sentence and paragraph nodes, as these contain the most specific information. Section and document nodes, while indexed, are too coarse for precise matching.

Cross-Query Reranking. Results from multiple queries are aggregated and reranked. While neural rerankers such as cross-encoders (Nogueira and Cho, 2019; Chen et al., 2024) and late interaction models (Khattab and Zaharia, 2020; Santhanam et al., 2022) offer strong performance, we employ a lightweight score-based reranking to minimize latency. For each unique node v appearing in $\bigcup_i \mathcal{R}_i$, we compute:

- **Frequency:** $f_v = |\{i : v \in \mathcal{R}_i\}|$, the number of queries that retrieved v
- **Total score:** $s_v = \sum_{i:v \in \mathcal{R}_i} \text{sim}(q_i, v)$, the cumulative similarity

We support three reranking strategies:

- **FREQUENCY:** Sort by (f_v, s_v) descending, so nodes retrieved by more queries rank higher
- **SCORE:** Sort by s_v descending, so nodes with highest cumulative similarity rank higher
- **COMBINED:** Sort by $\alpha \cdot \hat{f}_v + (1 - \alpha) \cdot \hat{s}_v$ where $\hat{\cdot}$ denotes min-max normalization

After reranking, results are optionally truncated to k_{final} nodes to control context length.

Hierarchical Context Expansion. Retrieved nodes are expanded with their hierarchical context. For each matched node v , we include its parent node (providing broader context) and sibling sentences (providing local context). This expansion leverages the tree structure: a matched sentence automatically brings its containing paragraph, enabling the LLM to understand the surrounding discussion without redundant retrieval. Since multi-query retrieval often returns the same node from different queries, we deduplicate by node ID after reranking, removing cross-query duplicates before context expansion.

Optional BM25 Augmentation. For queries requiring exact keyword matching (e.g., specific model names or numerical values), we optionally augment dense retrieval with BM25 (Robertson and Zaragoza, 2009) results. While learned sparse methods like SPLADE (Formal et al., 2021) offer improved performance, we use traditional BM25 for simplicity. Hybrid sparse-dense retrieval has been shown to provide complementary benefits (Mandikal et al., 2024); the sparse results are appended after dense results without score fusion, providing complementary lexical matches.

4.3 ANSWERING PIPELINE

Prompt Construction. Retrieved snippets are formatted with explicit reference markers: `[ref_id={doc_id}] {text}`. This format enables the LLM to directly cite sources by document ID. For documents with images, captions are included in a separate “Referenced media” section.

We found that prompt ordering significantly affects answer quality. Placing the question and instructions *after* the context (rather than before) yields better results, consistent with the “lost in the middle” phenomenon (Liu et al., 2024). This effect arises from attention sink behavior (Xiao et al., 2024a), where initial tokens accumulate disproportionate attention mass due to softmax normalization, combined with recency bias from causal attention and rotary position encodings (Su et al., 2023).

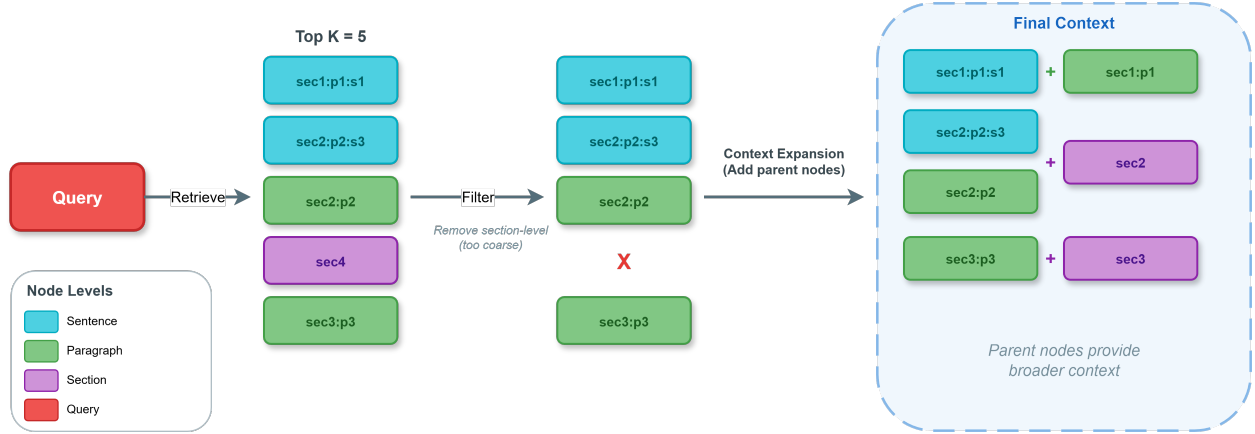


Figure 2: Hierarchical context expansion. Given a query, we retrieve Top-K nodes across multiple granularity levels (sentences in cyan, paragraphs in green, sections in purple). Section-level nodes are filtered out as too coarse (marked with X). The remaining nodes are expanded by adding their parent nodes to provide broader context. For example, a matched sentence `sec1:p1:s1` brings its parent paragraph `sec1:p1`, while a matched paragraph `sec2:p2` brings its parent section `sec2`.

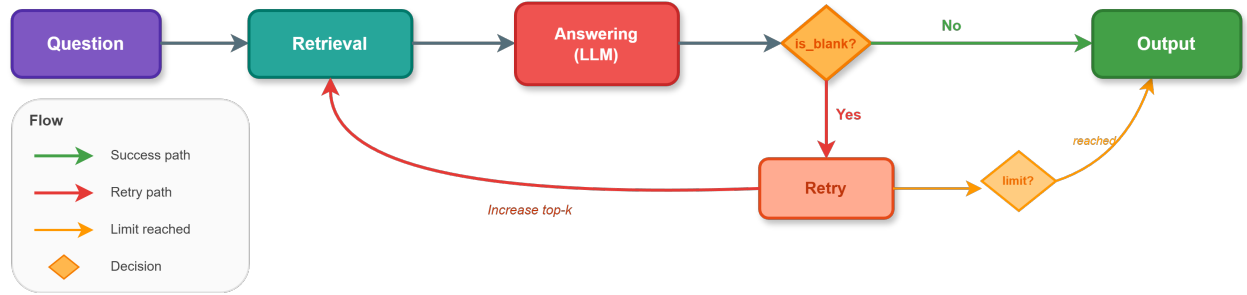


Figure 3: Retry mechanism flow. When the LLM outputs `is_blank=true` (insufficient evidence), the system increases top- k and re-retrieves context. This loop continues until a valid answer is produced or the retry limit is reached, at which point the system outputs a blank response.

Structured Output. The LLM is instructed to output JSON with four fields:

- **answer:** Natural language response
- **answer_value:** Extracted value (numeric or categorical)
- **ref_id:** List of source document IDs
- **explanation:** Reasoning chain (for debugging)

When evidence is insufficient, the model outputs `is_blank=true` to explicitly abstain rather than hallucinate.

Retry Mechanism. If the LLM indicates insufficient evidence (Figure 3), we retry with increased k to provide more context. This iterative deepening allows the system to find relevant information that may have ranked just below the initial cutoff.

4.4 ENSEMBLE INFERENCE

Single-inference answers exhibit variance due to LLM sampling stochasticity. Following self-consistency (Wang et al., 2023b) and recent work on LLM ensembles (Chen et al., 2025), we improve robustness through ensemble inference with majority voting.

Multi-Run Aggregation. For each question, we perform m independent inference runs with temperature > 0 . Each run produces an answer tuple $(a^{(j)}, R^{(j)})$ for $j \in \{1, \dots, m\}$.

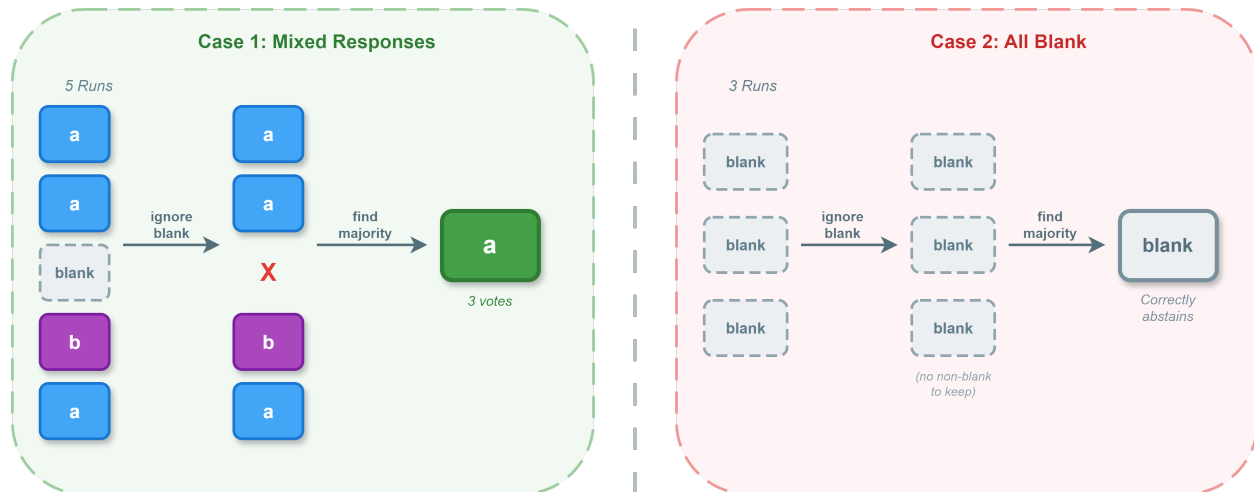


Figure 4: Ensemble voting with blank handling. **Case 1 (Mixed Responses)**: When non-blank answers exist among 5 runs $\{a, a, \text{blank}, b, a\}$, blank responses are filtered (X) before majority voting, yielding $\{a, a, b, a\}$ with majority “a” (3 votes). **Case 2 (All Blank)**: When all runs return blank, the system correctly abstains rather than hallucinating, preserving the hallucination score.

Voting Strategies. We support multiple voting modes:

- **INDEPENDENT**: Vote on answer and references separately
- **ANSWERPRIORITY**: First vote on answer, then collect references from matching runs
- **REFPRIORITY**: First vote on references, then select answer from matching runs
- **UNION**: Vote on answer, take union of references from matching runs
- **INTERSECTION**: Vote on answer, take intersection of references

Blank Handling. A critical design choice is handling abstention (`is_blank`) answers. Setting `ignore_blank=true` filters out blank answers before voting if any non-blank answer exists. This prevents conservative runs from dominating when evidence exists but is difficult to find. Figure 4 illustrates the two cases.

4.5 MULTIMODAL RETRIEVAL

5 EXPERIMENTS

We conduct ablation studies to evaluate each component of KohakuRAG on the WattBot 2025 Challenge training set.

5.1 EXPERIMENTAL SETTINGS

Dataset and Corpus. The WattBot 2025 corpus comprises 32 reference documents (research papers and technical slides) totaling approximately 500K tokens. The training set contains 41 questions with ground-truth answers and reference IDs, spanning AI energy characteristics including power consumption, water usage, carbon footprint, and performance metrics. After hierarchical parsing, our index contains 32 documents, 639 sections, 944 paragraphs, 24,565 sentences, and 280 images (275 with extractable content).

Evaluation Metrics. Following the official WattBot scoring protocol: **Value Score** (75%) measures answer accuracy with $\pm 0.1\%$ tolerance for numerics; **Reference Score** (15%) computes Jaccard similarity of reference IDs; **Hallucination Score** (10%) rewards correct abstention. Final score: $0.75 \times \text{value} + 0.15 \times \text{ref} + 0.10 \times \text{hallucination}$.

Default Configuration. Unless otherwise noted, we use GPT-oss-120B as the generator, Jina v3 embeddings (768-dim), top- $k=16$ per query, 4 planner queries, combined reranking, and top- $k_{\text{final}}=32$. Each

Technical documents often contain critical information in figures and tables (Cho et al., 2024; Yin et al., 2024). KohakuRAG supports two modes of multimodal retrieval: **Caption-Based Retrieval**. Image captions generated during indexing are stored as paragraph nodes. Text queries naturally retrieve relevant images through their captions, which are included in the LLM prompt as text. **Vision-Enabled Retrieval**. With Jina v4 embeddings, images can be directly embedded and retrieved, similar to ColPali (Faysse et al., 2025). For vision-capable LLMs such as GPT-4V, Gemini, and Qwen2-VL (Wang et al., 2024b; Liu et al., 2023), retrieved images are sent as base64-encoded content alongside text, enabling direct interpretation of visual information.

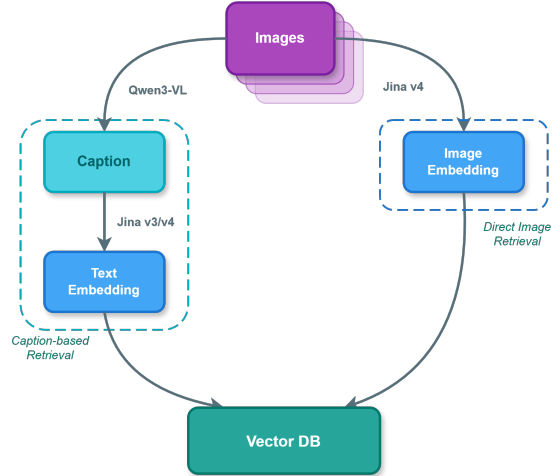


Figure 5: Dual-path image processing. **Left:** Caption-based retrieval via Qwen3-VL captioning and Jina v3/v4 text embedding. **Right:** Direct image embedding via Jina v4’s multimodal encoder.

Table 1: Retrieval component ablations (final score). Best per column in **bold**, second best underlined. Significance vs. best: $^{\dagger}p<0.10$, $^*p<0.05$, $^{**}p<0.01$.

Component	Setting	$k=4$	$k=8$	$k=16$	Avg Rank
Embedding	Jina v3 (text only)	0.752 ^{**}	0.801 [†]	0.824 [*]	3.00
	Jina v3 (+ captions)	0.859	<u>0.836</u> [†]	0.824 ^{**}	2.00
	Jina v4 (+ images)	0.869	0.868	0.890	1.00
Planner	2 queries	0.562 ^{**}	0.466 [*]	0.558 ^{**}	3.00
	4 queries	<u>0.678</u>	0.830	<u>0.811</u>	1.67
	6 queries	0.787	<u>0.809</u>	0.838	1.33
Reranking	None	0.808	0.765	0.833	1.67
	Frequency	0.396 [*]	0.533 ^{**}	0.585 ^{**}	4.00
	Score	0.605	<u>0.792</u>	0.785 [*]	2.67
	Combined	<u>0.798</u>	0.822	<u>0.814</u>	1.67

configuration is evaluated over 3 runs with different random seeds. Statistical significance is assessed using Welch’s t -test (unequal variances); we report $p<0.05$ as significant and $p<0.10$ as marginally significant given the inherent stochasticity of LLM sampling.

5.2 RETRIEVAL COMPONENT ANALYSIS

We evaluate embedding models, query planning, and reranking strategies. Table 1 presents results across retrieval depths $k \in \{4, 8, 16\}$.

Jina v4 with native image embeddings achieves the best retrieval quality (0.890 at $k=16$), outperforming text-only Jina v3 by 6.6 percentage points ($p=0.016$, paired t -test). Increasing planner queries from 2 to 6 improves performance substantially (0.562→0.787 at $k=4$, +40% relative), as diverse query formulations capture different aspects of the information need. For reranking, the COMBINED strategy provides the most consistent results across retrieval depths, while FREQUENCY-only yields substantially lower scores (0.396 vs 0.808 at $k=4$; $p<0.01$), demonstrating that retrieval scores contain important complementary information beyond frequency signals.

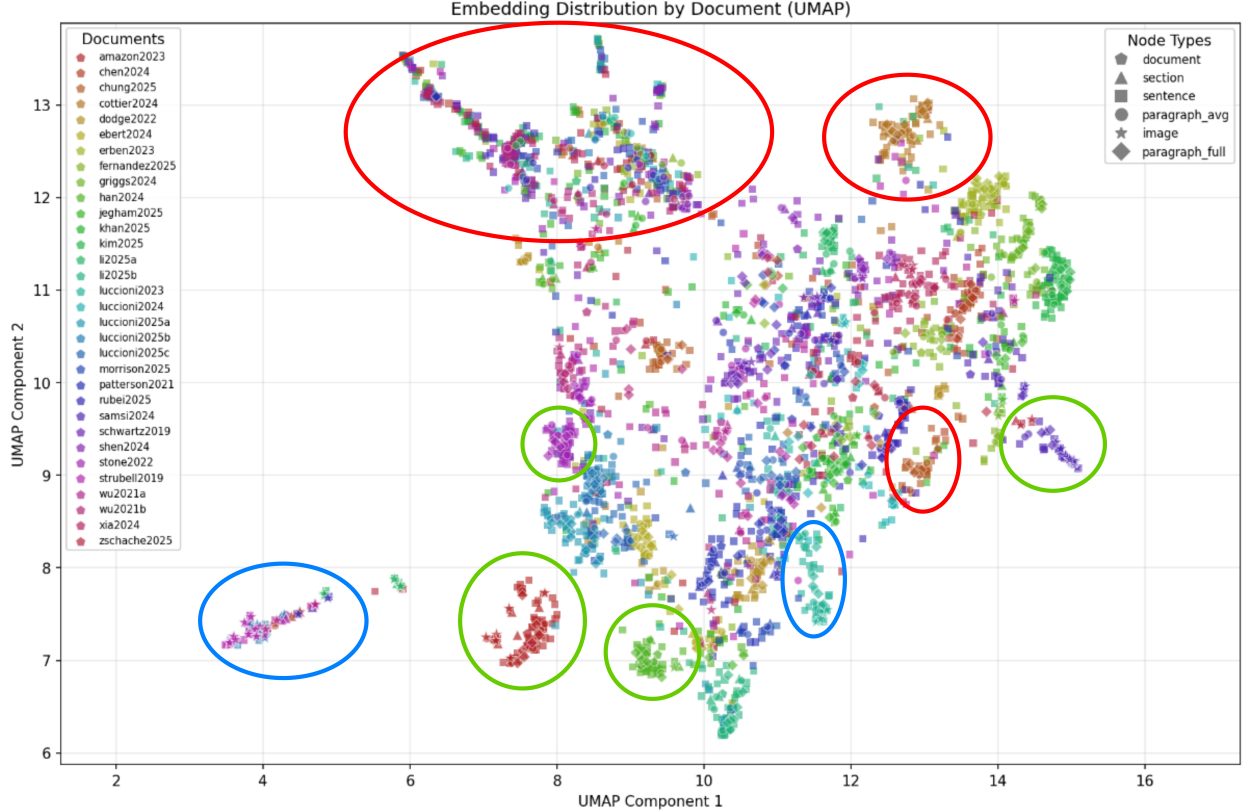


Figure 6: UMAP visualization of Jina v4 embeddings for all indexed nodes, colored by source document. Points are differentiated by node type (documents, sections, sentences, paragraph averages, images, and full paragraphs). We annotate three types of regions: **green circles** mark document-dominant clusters where one paper dominates 50–99% of nodes with minor contributions from related documents; **red circles** highlight cross-document semantic clusters where multiple documents mix, indicating shared topics (e.g., energy measurement methodologies); **blue circles** show image-text alignment regions where image embeddings (stars) cluster with semantically related sentences. This validates that our hierarchical embeddings preserve document coherence while enabling cross-document similarity matching.

Table 2: Context enhancement ablations (final score). Best per column in **bold**, second best underlined. Significance vs. best: $^{\dagger}p<0.10$, $^*p<0.05$, $^{**}p<0.01$, $^{***}p<0.001$.

Component	Setting	$k=4$	$k=8$	$k=16$	Avg Rank
Prompt Order	Standard (Q→C)	0.418 [*]	0.485 ^{***}	<u>0.704</u>	2.00
	Reordered (C→Q)	0.752	0.783	0.811	1.00
BM25 top- k	0 (dense only)	0.688 [*]	0.741	0.788	3.67
	2	0.701	0.822	<u>0.812</u>	2.00
	4	0.763	<u>0.801</u>	0.819	1.33
	8	<u>0.720</u>	0.720 ^{**}	0.806	3.00
Max Retries	0 (disabled)	0.488 ^{**}	<u>0.836</u>	<u>0.837</u>	3.00
	1	<u>0.720</u>	0.843	0.846	2.00
	2	0.827	0.854	0.871	1.00

5.3 CONTEXT ENHANCEMENT

Table 2 evaluates prompt reordering, BM25 hybrid retrieval, and the retry mechanism.

Prompt reordering yields substantial improvements (0.418→0.752 at $k=4$, +80% relative), confirming the “lost in the middle” phenomenon (Liu et al., 2024). Adding BM25 improves dense-only retrieval by 3.1 percentage points at $k=16$ (0.788→0.819). The retry mechanism provides +69% relative improvement at $k=4$ (0.488→0.827), effectively addressing abstention failures where the model incorrectly abstains due to insufficient initial context.

5.4 LLM COMPARISON

We compare seven LLMs using Jina v4 embeddings, including open-weight models like GPT-oss-120B (OpenAI, 2025b). Table 3 reports mean scores across 3 runs.

Table 3: LLM comparison (final score). Best in **bold**, second underlined. Significance: $^\dagger p<0.10$, $^* p<0.05$, $^{**} p<0.01$.

Model	$k=4$	$k=8$	$k=16$	Rank
GPT-5-nano	.783 ^{**}	.746 ^{**}	.787 ^{**}	7.00
GPT-5-mini	.812 [*]	.804 [*]	.847	4.67
GPT-oss-120B	.805 [†]	.763 [*]	.833 [*]	5.67
Mistral-large	.853 [*]	.862	.816 [†]	4.67
Gemini-3-pro	.871 [†]	.873	<u>.875</u>	2.67
Kimi-k2	<u>.872[*]</u>	.889	.874	2.00
Grok-4.1-fast	.908	<u>.885</u>	.888	1.33

Grok-4.1-fast achieves the best average rank (1.33) across all retrieval depths, with peak performance of 0.908 at $k=4$. The gap between Grok-4.1-fast and GPT-5-nano is highly significant ($p<0.001$), while differences among top-tier models are not statistically significant ($p>0.1$). Gemini-3-pro uniquely supports native image input. Smaller models (GPT-5-nano, avg rank 7.00) show significant degradation, indicating model capacity is critical.

5.5 ENSEMBLE INFERENCE

Table 4 presents ensemble experiments with varying sizes and voting strategies.

Ensemble performance exhibits logarithmic scaling with ensemble size, plateauing at $n=9-11$. The `ignore_blank` option consistently improves performance by 1.2 percentage points at $n=9$ (0.882→0.894) by excluding abstaining runs from voting. ANSWERPRIORITY voting achieves the best average rank (1.62) by ensuring reference consistency with the selected answer. Combining optimal configurations achieves **0.930** on the training set.

5.6 ERROR ANALYSIS

We analyzed 2,583 predictions across all LLM sweep runs to identify failure patterns. Of all predictions, 75.2% were fully correct. Among the 24.8% errors, the three dominant categories are: **unnecessary abstention** (26.8% of errors), where models output `is_blank` despite sufficient evidence; **reference mismatch** (23.6%), where answers are correct but citations are wrong; and **value selection errors** (22.2%), where models select incorrect but contextually plausible values. Rounding/calculation errors account for 13.3%, type mismatches for 12.2%, and unit conversion errors for only 1.6%, indicating that LLMs handle unit consistency well when the correct value is retrieved. Detailed categorization criteria and analysis are provided in Appendix C.

6 RESULTS

We evaluate KohakuRAG on the WattBot 2025 Challenge test set, which consists of 282 questions split into public (66%) and private (34%) partitions. KohakuRAG achieves **first place** on the final leaderboard with a private score of **0.861**. Table 5 summarizes our submitted configurations and compares public vs. private score dynamics across top teams.

Table 4: Ensemble analysis. Left: ignore_blank ablation (avg rank in parentheses); all differences for $n \geq 3$ are significant ($p < 0.01$). Right: voting strategy comparison at $n=15$; differences among top strategies are not significant ($p > 0.1$).

ignore_blank	$n=1$	$n=5$	$n=9$	$n=13$	$n=15$
Off (1.88)	.799	.866**	.882**	.883***	.884**
On (1.12)	.799	.886	.894	.896	.895
Δ	–	+0.020	+0.012	+0.013	+0.011

Voting	Score	Rank
AnswerPriority	0.892	1.62
Independent	0.889	1.75
Intersection	0.890	2.62
RefPriority	0.889	4.38
Union	0.884 [†]	4.62

Robustness of Ensemble Methods. Ensemble strategies provide substantially more robust predictions across evaluation partitions. The GPT-oss-120B 7-ensemble achieves the most stable performance (0.862 public, 0.857 private, a gap of only 0.5%). In contrast, single-run Gemini-3-pro exhibits high variance: while achieving the highest public score (0.901), its private score drops to 0.839 (−6.2%). This highlights a trade-off between peak performance and consistency, as single models may overfit to public partition characteristics, whereas ensembles generalize more reliably.

Leaderboard Dynamics. The public-to-private transition reveals instructive patterns. Teams ranking #21 and #20 publicly achieved #2 and #3 privately (+0.055, +0.044), suggesting their methods generalized better despite lower public scores. Conversely, Public #2 and #3 fell to Private #4 and #9 (−0.046, −0.037). KohakuRAG was the only submission to maintain first place across both partitions, validating our emphasis on ensemble diversity and hierarchical retrieval robustness.

Single-Model vs. Ensemble Performance. Gemini-3-pro with direct image input achieves the highest public score (0.901) in a single inference run, surpassing GPT-oss-120B ensembles. This demonstrates the value of native visual reasoning for documents containing figures, tables, and diagrams, as the model can directly interpret visual elements rather than relying on caption intermediation. However, the 6.2% degradation on the private partition suggests that single-model predictions are more susceptible to distribution shift between question subsets.

Hybrid Retrieval Impact. Adding BM25 sparse retrieval to the GPT-oss-120B ensemble improves private score stability. The 7-ensemble with BM25 achieves 0.857 on the private partition, outperforming the 5-ensemble without BM25 (0.852). This validates the complementary benefits of lexical matching for queries containing specific identifiers or technical terms that may not be captured by dense semantic similarity alone.

Cross-Model Ensemble. Our winning submission combines predictions from top-5 configurations spanning multiple model families (Gemini-3-pro, GPT-oss-120B). This cross-model ensemble achieves 0.861 on the private partition, suggesting that different model families capture complementary aspects of the document corpus. The ensemble aggregates Gemini-3-pro’s visual reasoning capabilities with GPT-oss-120B’s text comprehension strengths, achieving robust performance across diverse question types while mitigating individual model biases.

Table 5: WattBot 2025 test set results and leaderboard comparison. **Left:** Our submitted configurations (all use Jina v4 with hierarchical indexing). **Right:** Public vs. private score dynamics across top teams. Only KohakuRAG maintained #1 on both partitions.

Model	Config	n	Pub	Priv
GPT-oss-120B	No img/BM25	5	.884	.852
GPT-oss-120B	+BM25	7	.862	.857
Gemini-3-pro	+Images	1	.901	.839
Top-5 mixed ensemble		–	.902	.861

Team	Public	Private	Δ
Ours	.902 (#1)	.861 (#1)	−.041
Private #2	.803 (#21)	.858 (#2)	+0.055
Private #3	.807 (#20)	.851 (#3)	+0.044
Public #2	.886 (#2)	.840 (#4)	−.046
Public #3	.868 (#3)	.831 (#9)	−.037

7 CONCLUSION

We presented KohakuRAG, a hierarchical RAG framework that addresses three limitations of standard RAG for document-grounded question answering: structure loss from flat chunking (via four-level tree indexing with bottom-up embedding propagation), query-document vocabulary mismatch (via LLM-powered multi-query planning with cross-query reranking), and answer instability from single-inference stochasticity (via ensemble voting with abstention-aware blank filtering). Ablation studies show that prompt ordering has outsized impact (+80% from context-before-question placement), that the retry mechanism effectively addresses unnecessary abstention (the dominant error mode at 26.8% of failures) with +69% relative improvement, and that hierarchical dense retrieval alone suffices for citation-heavy tasks (BM25 adds only +3.1pp). Cross-model ensembles provide robustness against distribution shift between evaluation partitions. On the WattBot 2025 Challenge, KohakuRAG achieved first place on both the public and private leaderboards with a final score of 0.861. Notably, we were the only team to maintain the top position across both partitions while other competitive submissions showed score degradation up to -0.046 .

Limitations. Several limitations warrant future investigation. Our rule-based document parsing relies on heuristics for section detection and may fail on irregular layouts. Reference mismatch errors (23.6% of failures) indicate that LLMs struggle with precise attribution even when answers are correct, often citing survey papers rather than original sources. Ensemble inference incurs linear cost scaling with the number of runs. The retry mechanism improves recall but increases latency for difficult questions. Promising directions include learned document parsing, reference-constrained decoding, and confidence-based adaptive retrieval that predicts when additional context is needed.

ACKNOWLEDGEMENTS

We thank Comfy Org for supporting the LLM inference costs used in this work.

REFERENCES

- Anthropic. Claude opus 4.5 model card and system card. Technical report, Anthropic, 2025. URL <https://www.anthropic.com/claude/opus>.
- Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection, 2023. URL <https://arxiv.org/abs/2310.11511>.
- Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond, 2023. URL <https://arxiv.org/abs/2308.12966>.
- Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-VL technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.

- Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation, 2024. URL <https://arxiv.org/abs/2402.03216>.
- Zhijun Chen et al. Harnessing multiple large language models: A survey on LLM ensemble, 2025. URL <https://arxiv.org/abs/2502.18036>.
- Jaemin Cho, Debanjan Mahata, Ozan Irsoy, Yujie He, and Mohit Bansal. M3DocRAG: Multi-modal retrieval is what you need for multi-page multi-document understanding, 2024. URL <https://arxiv.org/abs/2411.04952>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pages 4171–4186. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1423. URL <https://doi.org/10.18653/v1/n19-1423>.
- Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library, 2025. URL <https://arxiv.org/abs/2401.08281>.
- Christopher Endemann, Dhruva Jyoti Paul, and Annie Zhao. Wattbot 2025. <https://kaggle.com/competitions/WattBot2025>, 2025. Kaggle.
- Kenneth Enevoldsen et al. MMTEB: Massive multilingual text embedding benchmark, 2025. URL <https://arxiv.org/abs/2502.13595>.
- Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. ColPali: Efficient document retrieval with vision language models. In *International Conference on Learning Representations*, 2025. URL <https://arxiv.org/abs/2407.01449>.
- Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: sparse lexical and expansion model for first stage ranking. In *SIGIR '21: The 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, Virtual Event, Canada, July 11-15, 2021*, pages 2288–2292. ACM, 2021. doi: 10.1145/3404835.3463098. URL <https://doi.org/10.1145/3404835.3463098>.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024. URL <https://arxiv.org/abs/2312.10997>.
- Yunfan Gao et al. Retrieval-augmented generation: A comprehensive survey of architectures, enhancements, and robustness frontiers, 2025. URL <https://arxiv.org/abs/2506.00054>.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, et al. Gemma 3 technical report, 2025. URL <https://arxiv.org/abs/2503.19786>.
- Google DeepMind. Gemini 3 technical report. Technical report, Google DeepMind, 2025. URL <https://deepmind.google/models/gemini/pro/>.
- Michael Günther, Saba Sturua, Mohammad Kalim Akram, Isabelle Mohr, Andrei Ungureanu, Bo Wang, Sedigheh Eslami, Scott Martens, Maximilian Werk, Nan Wang, and Han Xiao. jina-embeddings-v4: Universal embeddings for multimodal multilingual retrieval, 2025. URL <https://arxiv.org/abs/2506.18902>.

- Michael Günther et al. Late chunking: Contextual chunk embeddings using long-context embedding models, 2024. URL <https://arxiv.org/abs/2409.04701>.
- Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions, 2023. URL <https://arxiv.org/abs/2311.05232>.
- Yizheng Huang and Jimmy Huang. A survey on retrieval-augmented text generation for large language models, 2024. URL <https://arxiv.org/abs/2404.10981>.
- Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning, 2022. URL <https://arxiv.org/abs/2112.09118>.
- Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky. Query expansion by prompting large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. URL <https://arxiv.org/abs/2305.03653>.
- Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data*, 7(3):535–547, 2021. doi: 10.1109/TBDATA.2019.2921572. URL <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1601–1611. Association for Computational Linguistics, 2017. URL <https://aclanthology.org/P17-1147/>.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, EMNLP 2020, Online, November 16-20, 2020*, pages 6769–6781. Association for Computational Linguistics, 2020. doi: 10.18653/V1/2020.EMNLP-MAIN.550. URL <https://doi.org/10.18653/v1/2020.emnlp-main.550>.
- Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2020, Virtual Event, China, July 25-30, 2020*, pages 39–48. ACM, 2020. doi: 10.1145/3397271.3401075. URL <https://doi.org/10.1145/3397271.3401075>.
- Aditya Kusupati, Gantavya Bhatt, Aniket Rege, Matthew Wallingford, Aditya Sinha, Vivek Ramanujan, William Howard-Snyder, Kaifeng Chen, Sham Kakade, Prateek Jain, and Ali Farhadi. Matryoshka representation learning, 2022. URL <https://arxiv.org/abs/2205.13147>.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. URL <https://aclanthology.org/Q19-1026/>.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- Zehan Li, Xin Zhang, Yanzhao Zhang, Dingkun Long, Pengjun Xie, and Meishan Zhang. Towards general text embeddings with multi-stage contrastive learning, 2023. URL <https://arxiv.org/abs/2308.03281>.
- Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning, 2023. URL <https://arxiv.org/abs/2304.08485>.

- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173, 2024. doi: 10.1162/tacl.a_00638. URL <https://aclanthology.org/2024.tacl-1.9/>.
- Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. Query rewriting for retrieval-augmented large language models, 2023. URL <https://arxiv.org/abs/2305.14283>.
- Yu. A. Malkov and D. A. Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, 2018. URL <https://arxiv.org/abs/1603.09320>.
- Priyanka Mandikal et al. Sparse meets dense: A hybrid approach to enhance scientific document retrieval, 2024. URL <https://arxiv.org/abs/2401.04055>.
- Mistral AI. Introducing Mistral 3. Technical report, Mistral AI, 2025. URL <https://mistral.ai/news/mistral-3>.
- Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark, 2023. URL <https://arxiv.org/abs/2210.07316>.
- Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. In *arXiv preprint arXiv:1901.04085*, 2019. URL <https://arxiv.org/abs/1901.04085>.
- OpenAI. GPT-5 system card. Technical report, OpenAI, 2025a. URL <https://cdn.openai.com/gpt-5-system-card.pdf>.
- OpenAI. gpt-oss-120b & gpt-oss-20b model card. Technical report, OpenAI, 2025b. URL <https://openai.com/index/gpt-oss-model-card/>.
- Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*, pages 3980–3990. Association for Computational Linguistics, 2019. doi: 10.18653/V1/D19-1410. URL <https://doi.org/10.18653/v1/D19-1410>.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- Jon Saad-Falcon, Joe Barrow, Alexa Siu, Ani Nenkova, David Seunghyun Yoon, Ryan A. Rossi, and Franck Dernoncourt. PDFTriage: Question answering over long, structured documents, 2023. URL <https://arxiv.org/abs/2309.08872>.
- Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. ColBERTv2: Effective and efficient retrieval via lightweight late interaction. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 3715–3734. Association for Computational Linguistics, 2022. URL <https://arxiv.org/abs/2112.01488>.
- Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic RAG, 2025. URL <https://arxiv.org/abs/2501.09136>.
- Saba Sturua, Isabelle Mohr, Mohammad Kalim Akram, Michael Günther, Bo Wang, Markus Krimmel, Feng Wang, Georgios Mastrapas, Andreas Koukounas, Nan Wang, and Han Xiao. jina-embeddings-v3: Multilingual embeddings with task LoRA, 2024. URL <https://arxiv.org/abs/2409.10173>.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding, 2023. URL <https://arxiv.org/abs/2104.09864>.

- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.
- Liang Wang, Nan Yang, and Furu Wei. Query2doc: Query expansion with large language models, 2023a. URL <https://arxiv.org/abs/2303.07678>.
- Liang Wang, Nan Yang, Xiaolong Huang, Linjun Yang, Rangan Majumder, and Furu Wei. Improving text embeddings with large language models, 2024a. URL <https://arxiv.org/abs/2401.00368>.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Yang Fan, Kai Dang, Mengfei Du, Xuancheng Ren, Rui Men, Dayiheng Liu, Chang Zhou, Jingren Zhou, and Junyang Lin. Qwen2-VL: Enhancing vision-language model’s perception of the world at any resolution, 2024b. URL <https://arxiv.org/abs/2409.12191>.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023b. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- xAI. Grok 4.1 model card. Technical report, xAI, 2025. URL <https://data.x.ai/2025-11-17-grok-4-1-model-card.pdf>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2024a. URL <https://arxiv.org/abs/2309.17453>.
- Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packed resources for general chinese embeddings, 2024b. URL <https://arxiv.org/abs/2309.07597>.
- Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=zeFrfgYzln>.
- Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation, 2024. URL <https://arxiv.org/abs/2401.15884>.
- Shukang Yin, Chaoyou Fu, Sirui Zhao, Ke Li, Xing Sun, Tong Xu, and Enhong Chen. A survey on multimodal large language models, 2024. URL <https://arxiv.org/abs/2306.13549>.
- Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. Retrieval-augmented generation for AI-generated content: A survey, 2024. URL <https://arxiv.org/abs/2402.19473>.

Appendix

TABLE OF CONTENTS

A	Implementation Details	20
A.1	Indexing Pipeline	20
A.2	Retrieval Pipeline	21
A.3	Answer Generation Pipeline	23
A.4	Hyperparameter Configuration	25
B	Complete Sweep Results	25
B.1	Experimental Setup	25
B.2	Embedding and Multimodal Configuration	25
B.3	Image Retrieval Analysis	26
B.4	Retrieval Configuration	27
B.5	Context Enhancement	28
B.6	LLM Comparison	28
B.7	Ensemble Size Scaling	28
C	Error Analysis	29
C.1	Methodology and Error Categorization	29
C.2	Error Distribution and Patterns	30
C.3	Concrete Examples and Analysis	31
C.4	Implications for RAG System Design	33
D	WattBot 2025 Leaderboard Analysis	33
D.1	Final Leaderboard	33
D.2	Analysis Methodology	34
D.3	Score Transition Analysis	34
D.4	Rank Movement Patterns	36
D.5	KohakuRAG Performance and Implications	37
E	Limitations	38
F	Future Work	38

A IMPLEMENTATION DETAILS

This appendix provides comprehensive implementation details for KohakuRAG, including document parsing, embedding pipelines, storage, and inference configurations.

A.1 INDEXING PIPELINE

This section describes the document parsing, embedding computation, and vector storage components that transform raw documents into a searchable hierarchical index.

Document Parsing. Document parsing employs a rule-based pipeline tailored for technical PDFs and slide decks. For PDF documents, we use PyMuPDF to extract text with position information, then apply heuristics based on font size, spacing, and formatting to identify section boundaries. Slide decks are parsed page-by-page, treating each slide as a section. The parser outputs a JSON tree structure where each node contains:

- **id:** Hierarchical identifier (e.g., `doc1:sec2:p3:s4`)
- **level:** One of `{document, section, paragraph, sentence}`
- **content:** Raw text content
- **children:** List of child node IDs
- **metadata:** Source file, page number, bounding box (for images)

PDF Text Extraction. Text extraction uses PyMuPDF’s `get_text("dict")` method, which returns structured information including bounding boxes, font sizes, and font names. We use the following heuristics:

- **Section detection:** Text blocks with font size $\geq 1.2 \times$ the median font size are treated as potential section headers. Numbered patterns (e.g., “1.2”, “A.3”) further confirm section boundaries.
- **Paragraph detection:** Consecutive text blocks with vertical spacing $< 1.5 \times$ line height are merged into paragraphs. Larger gaps indicate paragraph boundaries.
- **Sentence segmentation:** We use NLTK’s `sent_tokenize` with custom rules for handling common abbreviations in technical text (e.g., “Fig.”, “et al.”, “i.e.”).

Image Extraction. Image extraction uses PyMuPDF’s `get_images()` to identify embedded figures, which are saved as PNG files. We handle PDF soft masks (SMask) correctly to preserve transparency for diagrams and charts. Captions are generated via Qwen3-VL with a prompt requesting concise descriptions of technical content, data values, and axis labels for charts:

Describe this image from a technical document.

Focus on:

1. Data values and numerical information
2. Axis labels and units for charts
3. Key visual elements and their relationships

Keep the description concise (2-3 sentences).

Embedding Pipeline. The embedding pipeline processes nodes bottom-up through the hierarchy. For Jina v3, we use the official API with `task=retrieval.passage` for indexing and `task=retrieval.query` for queries. For Jina v4, we use `jina-clip-v2` which natively supports both text and image inputs.

Algorithm 2 presents the bottom-up embedding propagation procedure.

Algorithm 2 Bottom-Up Embedding Propagation**Require:** Document tree $T = (V, E)$, encoder E **Ensure:** Embeddings $\{\mathbf{e}_v\}_{v \in V}$

```

1: function EMBEDTREE( $T, E$ )
2:    $\mathcal{L} \leftarrow \{v \in V : \mathcal{C}(v) = \emptyset\}$  ▷ Collect leaf nodes
3:   for  $v \in \mathcal{L}$  do ▷ Embed leaves (batched)
4:      $\mathbf{e}_v \leftarrow E(t_v)$ 
5:   end for

6:   for level  $\in [\text{paragraph}, \text{section}, \text{document}]$  do ▷ Bottom-up
7:     for  $v$  at current level do
8:        $\mathbf{e}_v \leftarrow \frac{\sum_{c \in \mathcal{C}(v)} |t_c| \cdot \mathbf{e}_c}{\sum_{c \in \mathcal{C}(v)} |t_c|}$  ▷ Weighted average
9:     end for
10:  end for
11:  return  $\{\mathbf{e}_v\}_{v \in V}$ 
12: end function

```

Embedding Dimensions. Jina v4 supports Matryoshka representations with configurable output dimensions (128–2048). We use 512 dimensions as the default, balancing retrieval quality with storage efficiency. Matryoshka truncation is applied post-embedding by slicing vectors to the target dimension.

Vector Storage. We use sqlite-vec (v0.1.1), a SQLite extension providing approximate nearest neighbor search via IVF-PQ indexing. The schema stores:

- **nodes** table: node metadata (id, level, content, parent_id)
- **embeddings** virtual table: vector index with node_id foreign key

KohakuVault wraps sqlite-vec with a Python interface supporting batch insert, filtered search (by level), and automatic index rebuilding when the corpus changes.

A.2 RETRIEVAL PIPELINE

This section describes the query planning, cross-query reranking, and optional BM25 hybrid retrieval components that transform user questions into ranked document passages.

Query Planning. The query planner uses a structured prompt requesting n diverse queries:

```

Given the question: {question}
Generate {n} search queries that:
1. Rephrase using alternative terminology
2. Expand abbreviations (e.g., PUE -> Power Usage Effectiveness)
3. Add likely keywords from technical documents
4. Decompose into sub-questions if compound
Output as JSON array of strings.

```

The planner LLM (same as generator) returns queries parsed via JSON; malformed outputs fall back to the original question only.

Example Query Expansion. For the question “What is the PUE of Google’s data centers?”, the planner generates:

1. “Google data center Power Usage Effectiveness”
2. “Google infrastructure energy efficiency ratio”
3. “PUE metric Google cloud computing facilities”
4. “Google sustainability report data center efficiency”

Cross-Query Reranking. Algorithm 3 presents the cross-query reranking procedure that aggregates results from multiple planned queries.

Algorithm 3 Cross-Query Reranking

Require: Query results $\{\mathcal{R}_i\}_{i=1}^n$ where $\mathcal{R}_i = \{(v, s_{iv})\}$, strategy $\in \{\text{FREQ}, \text{SCORE}, \text{COMBINED}\}$

Ensure: Ranked list of nodes

```

1: function RERANK( $\{\mathcal{R}_i\}$ , strategy)
2:    $\mathcal{V} \leftarrow \bigcup_i \{v : (v, s) \in \mathcal{R}_i\}$  ▷ Unique nodes
3:   for  $v \in \mathcal{V}$  do
4:      $f_v \leftarrow |\{i : v \in \mathcal{R}_i\}|$  ▷ Frequency
5:      $s_v \leftarrow \sum_{i: v \in \mathcal{R}_i} s_{iv}$  ▷ Total score
6:   end for

7:   if strategy = FREQUENCY then
8:     return SORTBY( $\mathcal{V}, (f_v, s_v), \text{desc}$ )
9:   else if strategy = SCORE then
10:    return SORTBY( $\mathcal{V}, s_v, \text{desc}$ )
11:  else if strategy = COMBINED then
12:     $\hat{f}_v \leftarrow (f_v - \min f) / (\max f - \min f)$  ▷ Normalize
13:     $\hat{s}_v \leftarrow (s_v - \min s) / (\max s - \min s)$ 
14:    return SORTBY( $\mathcal{V}, 0.5 \cdot \hat{f}_v + 0.5 \cdot \hat{s}_v, \text{desc}$ )
15:  end if
16: end function

```

BM25 Indexing. BM25 indexing uses rank_bm25 library with default parameters ($k_1 = 1.5$, $b = 0.75$). We index paragraph-level content only (not sentences) to balance precision with recall. Tokenization uses NLTK word.tokenize with lowercasing and stopword removal.

Hybrid Retrieval. At query time, BM25 scores are computed for all paragraphs, and top- k_{bm25} results are appended to dense retrieval results without score normalization. This simple concatenation strategy avoids the need to calibrate score distributions between dense and sparse retrievers.

Tree Deduplication. Because retrieval targets both sentence and paragraph nodes, multi-query retrieval can return a parent node alongside one or more of its descendants, e.g., paragraph `doc:sec1:p2` and sentence `doc:sec1:p2:s3`. Standard node-ID deduplication only removes exact duplicates and does not detect this hierarchical overlap; the child’s text is already contained within the parent, wasting context window capacity. Tree deduplication addresses this by removing any node whose ancestor (determined by node-ID prefix matching) is also present in the result set. This step is applied after reranking but before top- k_{final} truncation, so that the freed slots can be filled by other unique content.

Algorithm 4 presents the tree deduplication procedure. The algorithm first collects all unique node IDs from the expanded snippet set, then identifies nodes whose ancestor is also present by checking node-ID prefix relationships. Finally, it removes these subsumed nodes and deduplicates by node ID (keeping the first occurrence) in a single pass.

Algorithm 4 Tree Deduplication of Context Snippets**Require:** Expanded snippet set $\mathcal{S} = \{(v_i, t_i, s_i)\}$ where v_i is node ID, t_i is text, s_i is score**Ensure:** Deduplicated snippets $\mathcal{S}'$ with no ancestor–descendant overlap

```

1: function TREEDEDUP( $\mathcal{S}$ )
2:    $\mathcal{V} \leftarrow \{v_i : (v_i, t_i, s_i) \in \mathcal{S}\}$                                  $\triangleright$  Collect unique node IDs
3:    $\mathcal{D} \leftarrow \emptyset$                                                  $\triangleright$  IDs to discard
4:   for  $v \in \mathcal{V}$  do
5:     for  $u \in \mathcal{V} \setminus \{v\}$  do
6:       if  $v$  starts with  $u$ : then                                           $\triangleright u$  is ancestor of  $v$ 
7:          $\mathcal{D} \leftarrow \mathcal{D} \cup \{v\}$ ; break
8:       end if
9:     end for
10:  end for

11:   $\mathcal{S}' \leftarrow []$ ;  $seen \leftarrow \emptyset$ 
12:  for  $(v_i, t_i, s_i) \in \mathcal{S}$  do                                           $\triangleright$  Preserve original order
13:    if  $v_i \notin \mathcal{D}$  and  $v_i \notin seen$  then
14:      append  $(v_i, t_i, s_i)$  to  $\mathcal{S}'$ ;  $seen \leftarrow seen \cup \{v_i\}$ 
15:    end if
16:  end for
17:  return  $\mathcal{S}'$ 
18: end function

```

We evaluated the context reduction on the WattBot training set (41 questions) using Jina v4 embeddings with 4 planner queries $\times$ 16 retrievals per query and $\text{top-}k_{\text{final}}=32$. Table 6 reports aggregate statistics. Without deduplication, the $4 \times 16 = 64$ raw matches are truncated to 32, yielding 148K characters of context with substantial redundancy from nodes retrieved by multiple queries. Node-ID dedup removes these cross-query duplicates, reducing context by 35.0%. Tree dedup further removes hierarchical overlap, achieving a cumulative 51.4% reduction (from 148.1K to 72.0K characters) without discarding any unique information, since the parent node already subsumes all descendant text. Although tree deduplication was not enabled in our competition submissions, it provides a practical mechanism for reducing context length in deployment scenarios where LLM context windows or API costs are constrained.

Table 6: Context statistics under different deduplication modes (mean $\pm$ std over 41 training questions, 4 queries $\times$ 16 retrievals each). Node-ID dedup removes cross-query duplicates (−35.0%); tree dedup additionally removes hierarchical overlap (−51.4% cumulative).

Mode	Matches	Snippets	Context (chars)
No dedup	32.0 ± 0.0	64.0 ± 0.0	$148,125 \pm 34,415$
Node-ID dedup	20.9 ± 3.9	41.8 ± 7.8	$96,332 \pm 24,577$
Tree dedup	15.7 ± 4.4	31.4 ± 8.9	$71,956 \pm 23,449$

A.3 ANSWER GENERATION PIPELINE

This section describes the prompt construction, structured output format, and ensemble voting mechanism that transform retrieved context into final answers with citations.

Prompt Construction. Retrieved nodes are formatted as:

```
[ref_id={doc_id}] {node_content}
```

The full prompt structure (with reordering):

```
## Referenced Documents
{formatted_nodes}
```

```

## Referenced Media
{image_captions}

## Question
{question}

## Instructions
Answer the question using only the provided references.
If the answer is numeric, extract the exact value with units.
If evidence is insufficient, set is_blank=true.

Output JSON with fields:
- answer: Natural language answer
- answer_value: Extracted numeric or categorical value
- ref_id: List of source document IDs
- explanation: Brief reasoning (optional)
- is_blank: true if cannot answer from context

```

Context Length Management. We limit total context to 8,000 tokens to avoid degradation from overly long contexts. When retrieved content exceeds this limit, we truncate lower-ranked nodes while preserving the hierarchical context of top-ranked nodes.

Ensemble Voting. Ensemble voting aggregates m independent runs. For answer voting, we normalize answers (lowercase, strip whitespace, parse numerics) before counting. Algorithm 5 presents the complete voting procedure.

Algorithm 5 Ensemble Voting with Blank Handling

Require: Predictions $\mathcal{A} = \{(a^{(j)}, R^{(j)}, b^{(j)})\}_{j=1}^m$, `ignore_blank`, `vote_mode`

Ensure: Final prediction (a, R, b)

```

1: function ENSEMBLEVOTE( $\mathcal{A}$ , ignore_blank, vote_mode)
2:   if ignore_blank and  $\exists j : b^{(j)} = 0$  then
3:      $\mathcal{A} \leftarrow \{(a^{(j)}, R^{(j)}, b^{(j)}) \in \mathcal{A} : b^{(j)} = 0\}$  ▷ Filter blanks
4:   end if

5:   if  $\forall j : b^{(j)} = 1$  then ▷ All abstained
6:     return (null,  $\emptyset$ , 1)
7:   end if

8:    $\mathcal{G} \leftarrow \text{GROUPBY}(\mathcal{A}, \text{NORMALIZE}(a))$  ▷ Group by answer
9:    $a \leftarrow \arg \max_{g \in \mathcal{G}} |g|$  ▷ Plurality vote
10:   $\mathcal{A}_a \leftarrow \{(a', R', b') \in \mathcal{A} : \text{NORMALIZE}(a') = a\}$ 

11:  if vote_mode = ANSWERPRIORITY then
12:     $R \leftarrow \text{MAJORITYVOTE}(\{R' : (a', R', b') \in \mathcal{A}_a\})$ 
13:  else if vote_mode = UNION then
14:     $R \leftarrow \bigcup_{(a', R', b') \in \mathcal{A}_a} R'$ 
15:  else if vote_mode = INTERSECTION then
16:     $R \leftarrow \bigcap_{(a', R', b') \in \mathcal{A}_a} R'$ 
17:  end if
18:  return  $(a, R, 0)$ 
19: end function

```

Reference Aggregation Strategies. We support four strategies for combining reference sets across ensemble runs:

- **INDEPENDENT**: Majority vote on references separately from answer
- **ANSWERPRIORITY**: Collect refs only from runs matching winning answer
- **UNION**: Union of refs from runs matching winning answer
- **INTERSECTION**: Intersection of refs from runs matching winning answer

Numeric Answer Normalization. For numeric answers, we parse values and compare within a tolerance:

- Parse scientific notation (e.g., “1.5e6” $\rightarrow$ 1,500,000)
- Handle common units (e.g., “1.5 MW” $\rightarrow$ 1,500,000 W)
- Group answers within 0.1% relative difference as equivalent

A.4 HYPERPARAMETER CONFIGURATION

Table 7 summarizes the hyperparameter search space and defaults used throughout our experiments.

Table 7: Hyperparameter search space and defaults.

Parameter	Values Tested	Default	Notes
top- k (per query)	4, 8, 16	16	Higher = more coverage
top- k_{final}	16, 32, None	32	Post-rerank truncation
Planner queries	2, 4, 6	4	More = better coverage
Max retries	0, 1, 2	0	On abstention
BM25 top- k	0, 2, 4, 8	0	0 = dense only
Ensemble size	1, 3, 5, 7, 9, 11, 13, 15	1	Odd numbers only
Embedding dim (v4)	128, 256, 512, 1024, 2048	512	Matryoshka
Temperature	0.0, 0.3, 0.7, 1.0	0.7	For ensemble

B COMPLETE SWEEP RESULTS

This appendix presents comprehensive results from our hyperparameter sweeps, providing detailed empirical evidence supporting the design choices described in Section 5. Understanding these trade-offs is valuable for practitioners adapting KohakuRAG to different domains or computational constraints.

B.1 EXPERIMENTAL SETUP

All experiments were conducted on the WattBot 2025 training set (176 questions) using a consistent evaluation protocol. Each configuration reports mean score over 3 random seeds unless otherwise noted. Ensemble experiments use 32 stratified subsamples to estimate variance, with the same base predictions used across all ensemble sizes to enable fair comparison.

We organize results by component category: embedding configuration, image handling, retrieval parameters, context enhancement, LLM selection, and ensemble inference. For each category, we present both visualizations (figures) and detailed numerical results (tables) to support both quick intuition-building and precise comparison.

B.2 EMBEDDING AND MULTIMODAL CONFIGURATION

Figure 7 shows the interaction between LLM choice and embedding configuration. Jina v4 with native image embeddings consistently outperforms text-only Jina v3 across most LLMs, with the exception of GPT-oss-120B which shows degraded performance across all configurations (likely due to instruction-following limitations). Figure 8 demonstrates that the advantage of multimodal embeddings is most pronounced at lower retrieval depths ($k=4$), where precise retrieval is critical.

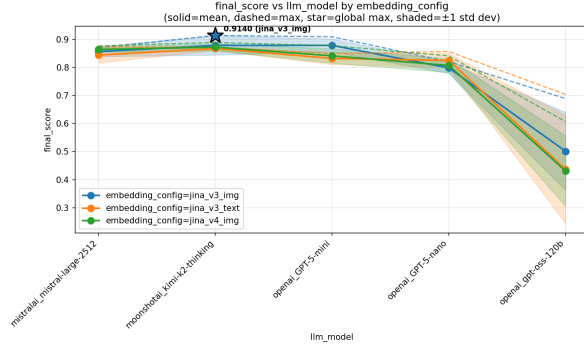


Figure 7: LLM × Embedding configuration.

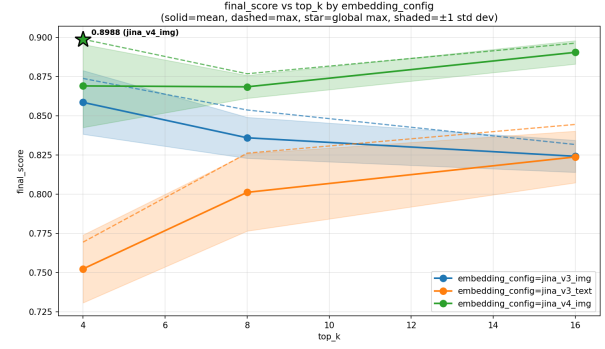
Figure 8: Top- k × Embedding configuration.

Table 8: LLM × Embedding configuration (final score).

Model	Jina v3 text	Jina v3 +img	Jina v4 +img
GPT-5-nano	0.825	0.798	0.807
GPT-5-mini	0.832	0.879	0.841
GPT-oss-120B	0.436	0.501	0.431
Mistral-large-2512	0.844	0.856	0.864
Kimi-k2-thinking	0.867	0.879	0.872

B.3 IMAGE RETRIEVAL ANALYSIS

We evaluate whether sending actual images (rather than just captions) to vision-capable LLMs improves answer quality. As shown in Figure 9 and Table 9, sending images provides modest improvement (+1.2% at $k=2$), but degrades with too many images ($k=8$), likely due to context dilution. This suggests a “sweet spot” exists for multimodal context: enough images to provide visual evidence, but not so many that they overwhelm textual reasoning.

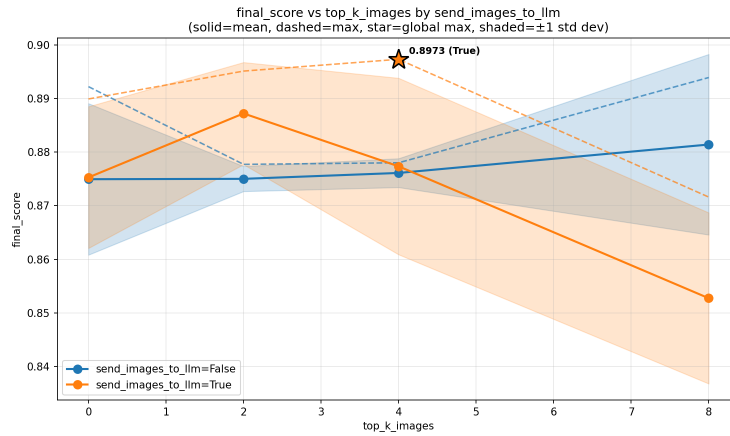
Figure 9: Send images to LLM × Image top- k configuration. Sending 2 images provides optimal performance.

Table 9: Image retrieval configuration with Grok-4.1-fast (final score).

Send to LLM	img $k=0$	img $k=2$	img $k=4$	img $k=8$
False (caption only)	0.875	0.875	0.876	0.881
True (with images)	0.875	0.887	0.877	0.853

B.4 RETRIEVAL CONFIGURATION

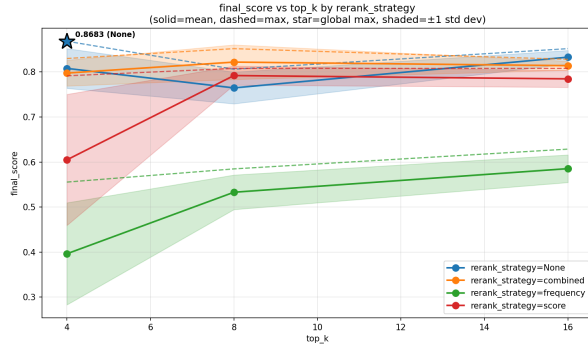
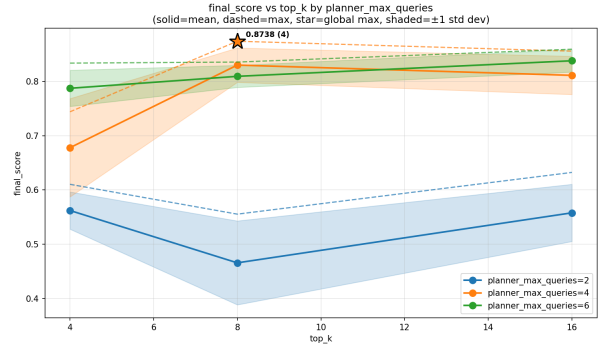
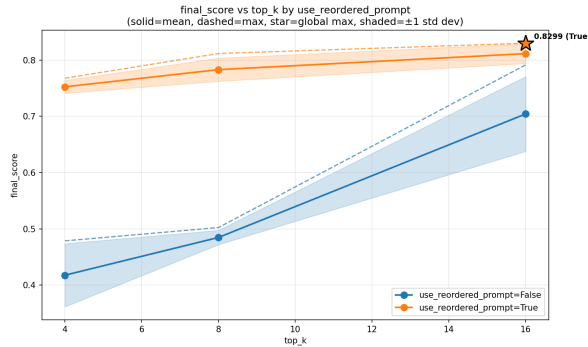
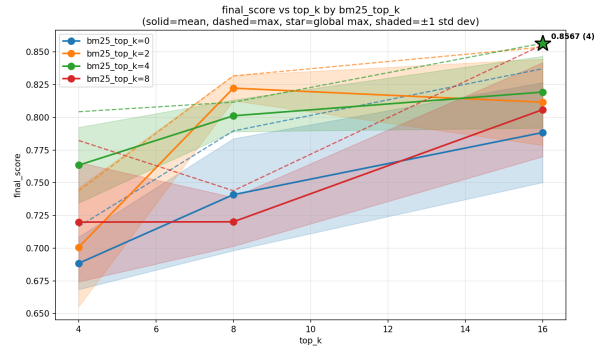
Figures 10–13 present ablations on retrieval parameters. Key findings:

Reranking Strategy (Figure 10). The COMBINED strategy (frequency + score) provides the most consistent results across retrieval depths. Pure frequency-based reranking severely underperforms, indicating that while document frequency provides useful signal, retrieval scores remain important.

Query Planning (Figure 11). Increasing planner queries from 2 to 6 improves performance substantially at all retrieval depths, with the largest gains at $k=4$ (+40%). More queries help capture diverse aspects of complex questions.

Prompt Reordering (Figure 12). Placing retrieved context before the question (C→Q order) dramatically outperforms the standard order, confirming the “lost in the middle” phenomenon. The effect is largest at lower k values where every piece of context matters.

BM25 Hybrid Search (Figure 13). Adding BM25 provides consistent improvements over dense-only retrieval, with optimal performance at BM25 $k=4$. Higher BM25 k values dilute the precision advantage.

Figure 10: Top- k × Reranking strategy.Figure 11: Planner queries × Top- k .Figure 12: Top- k × Prompt reordering.Figure 13: BM25 top- k × Dense top- k .

B.5 CONTEXT ENHANCEMENT

Figures 14–15 show the impact of retry mechanism and final context truncation.

Retry Mechanism (Figure 14). The retry mechanism provides substantial improvement, especially at low k values (+69% at $k=4$). When the model abstains due to insufficient evidence, expanding context and retrying often recovers the correct answer.

Final Top- k (Figure 15). Truncating reranked results to $\text{top-}k_{\text{final}}=32$ provides optimal balance between coverage and precision. No truncation (using all retrieved results) slightly degrades performance due to noise accumulation.

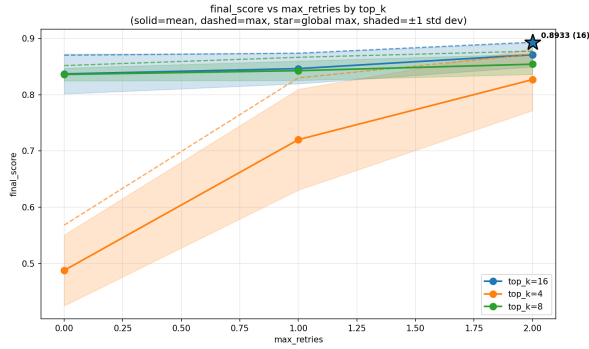


Figure 14: Top- $k \times$ Max retries.

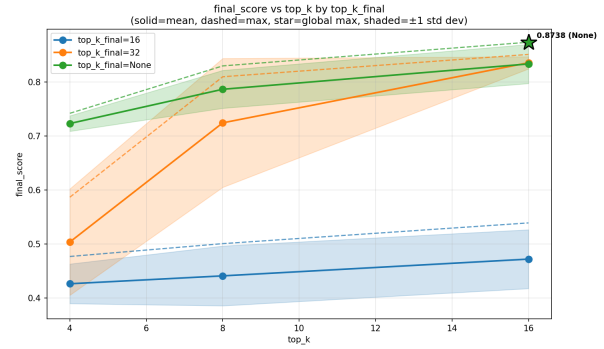


Figure 15: Top- $k \times$ Top- k_{final} .

B.6 LLM COMPARISON

Figure 16 presents a comprehensive comparison across seven LLMs. Grok-4.1-fast achieves the highest overall performance, with Kimi-k2-thinking and Gemini-3-pro as close seconds. Notably, performance rankings vary with retrieval depth: Grok excels at $k=4$, while Gemini shows more consistent performance across all k values.

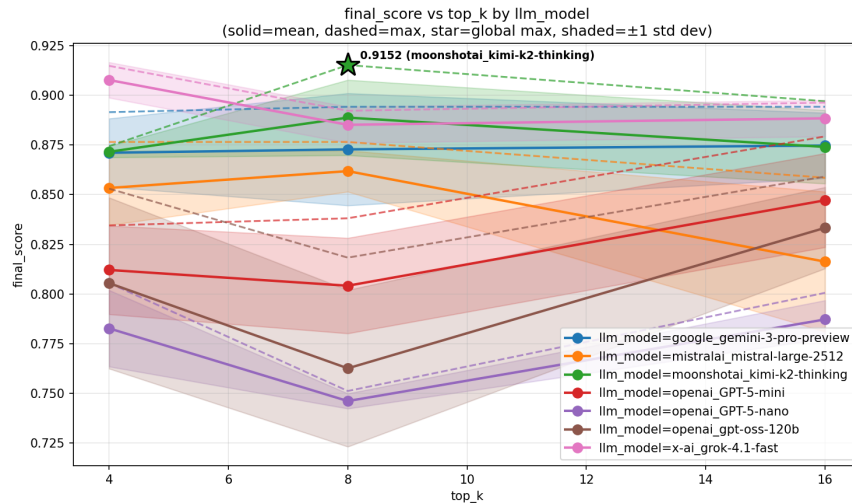


Figure 16: LLM model $\times$ Top- k configuration.

B.7 ENSEMBLE SIZE SCALING

Figures 17–18 and Tables 10–11 present ensemble analysis.

Ensemble Scaling (Figure 17). Performance exhibits logarithmic scaling with ensemble size, with diminishing returns beyond $n=9-11$. The `ignore_blank` option consistently improves performance by 1–2%, with larger gains at smaller ensemble sizes where individual blank answers have more impact.

Voting Strategies (Figure 18). ANSWERPRIORITY voting slightly outperforms other strategies, as it ensures reference consistency with the selected answer. UNION voting underperforms due to including spurious references from minority votes.

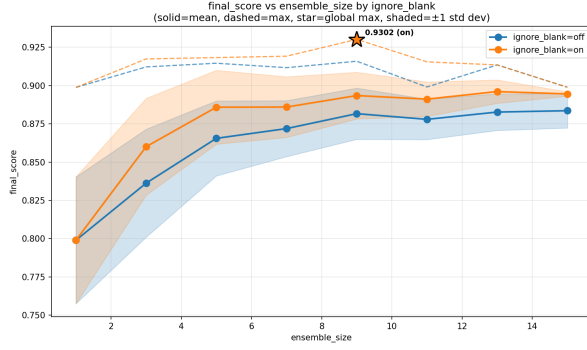
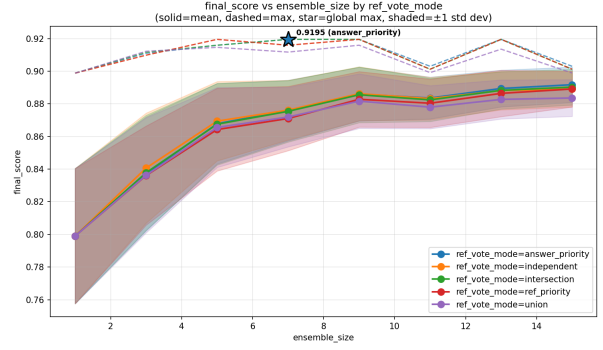
Figure 17: Ensemble size $\times$ ignore_blank.Figure 18: Ensemble size $\times$ Voting strategy.

Table 10: Ensemble size vs. voting strategy (final score, mean over 32 subsamples).

Strategy	$n=1$	$n=3$	$n=5$	$n=7$	$n=9$	$n=11$	$n=13$	$n=15$
Independent	.799	.841	.869	.876	.886	.883	.888	.889
RefPriority	.799	.836	.864	.871	.883	.880	.886	.889
AnswerPriority	.799	.837	.868	.876	.886	.884	.889	.892
Union	.799	.836	.866	.872	.882	.878	.883	.884
Intersection	.799	.838	.868	.875	.886	.882	.888	.890

Table 11: Effect of ignore_blank on ensemble performance (union voting).

ignore_blank	$n=1$	$n=3$	$n=5$	$n=7$	$n=9$	$n=11$	$n=13$	$n=15$
Off	.799	.836	.866	.872	.882	.878	.883	.884
On	.799	.860	.886	.886	.894	.891	.896	.895
Δ	–	+.024	+.020	+.014	+.012	+.013	+.013	+.011

C ERROR ANALYSIS

This appendix presents a comprehensive analysis of prediction errors across our hyperparameter sweeps, providing insights into the failure modes of RAG systems and informing future improvements. Understanding *why* systems fail is as important as measuring *how often* they succeed.

C.1 METHODOLOGY AND ERROR CATEGORIZATION

We analyzed 2,583 predictions across 63 sweep runs ($7 \text{ LLMs} \times 3 \text{ top-}k \text{ settings} \times 3 \text{ random seeds}$) to categorize failure modes. Each prediction was compared against ground truth using an automated classification pipeline. Of all predictions, 75.2% were fully correct, with the remaining 24.8% (641 errors) exhibiting various error types. Table 12 provides a high-level summary.

Table 12: Summary of error analysis across all sweep configurations.

Metric	Value
Total predictions analyzed	2,583
Correct predictions	1,942 (75.2%)
Total errors	641 (24.8%)
LLM configurations tested	7
Top- k settings tested	3 (4, 8, 16)
Random seeds per config	3

Classification Pipeline. We developed an automated error classification that assigns each incorrect prediction to exactly one category based on a deterministic decision tree:

Step 1 – Abstention Check: If ground truth is blank but prediction is not → **False positive**; if ground truth is not blank but prediction is blank → **False negative**; if both are blank → **Correct**.

Step 2 – Value Comparison: For non-blank predictions, we compute percentage error $\epsilon = |v_{\text{pred}} - v_{\text{true}}| / |v_{\text{true}}| \times 100\%$ and ratio $r = v_{\text{pred}} / v_{\text{true}}$ for numeric values; case-insensitive comparison for categorical values.

Step 3 – Numeric Error Classification: For $\epsilon > 0.1\%$ (WattBot tolerance): (1) **Unit conversion** if r is within $\pm 5\%$ of any power of 10; (2) **Rounding/calculation** if $\epsilon \leq 10\%$; (3) **Value selection** otherwise.

Step 4 – Reference and Type Errors: If value is correct but references differ → **Reference mismatch**. If expected numeric but got text (or vice versa) → **Type mismatch**. If both non-numeric text but different → **Categorical mismatch**.

C.2 ERROR DISTRIBUTION AND PATTERNS

Applying this classification to all 641 errors yields the distribution shown in Figure 19. The three dominant categories, unnecessary abstention (26.8%), reference mismatch (23.6%), and value selection (22.2%), together account for 72.6% of all errors, suggesting focused intervention points for system improvement.

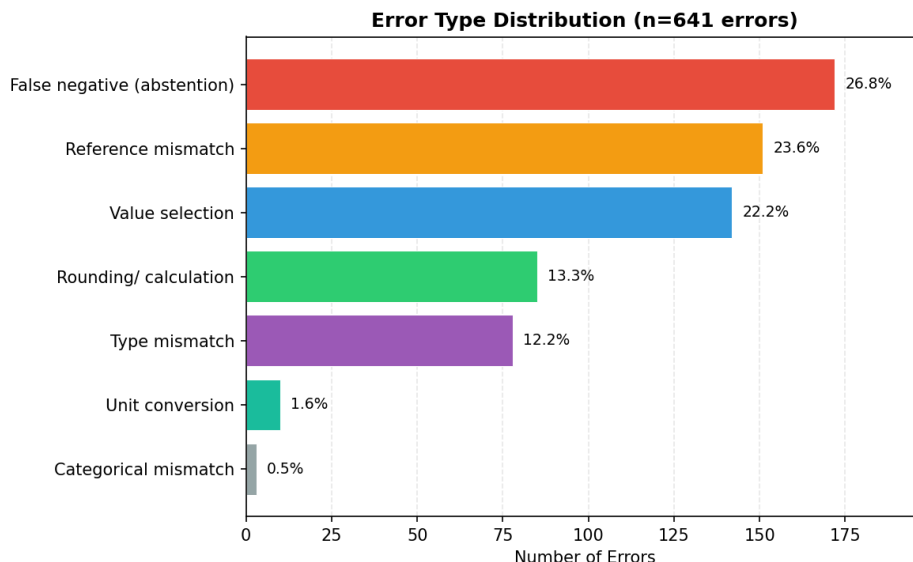


Figure 19: Distribution of error types across 641 errors from 63 sweep runs. The top three categories (abstention, reference mismatch, value selection) account for nearly three-quarters of all errors.

- **False negative (unnecessary abstention):** 172 errors (26.8%)
- **Reference ID mismatch:** 151 errors (23.6%)

- **Value selection errors:** 142 errors (22.2%)
- **Rounding/calculation errors:** 85 errors (13.3%)
- **Type mismatch:** 78 errors (12.2%)
- **Unit conversion errors:** 10 errors (1.6%)
- **Categorical mismatch:** 3 errors (0.5%)

Patterns by Configuration. Beyond aggregate statistics, we examine how error rates vary across LLM models and retrieval depths. Figure 20 shows the overall error rate (computed as $1 - \text{final_score}$) for each LLM and top- k combination.

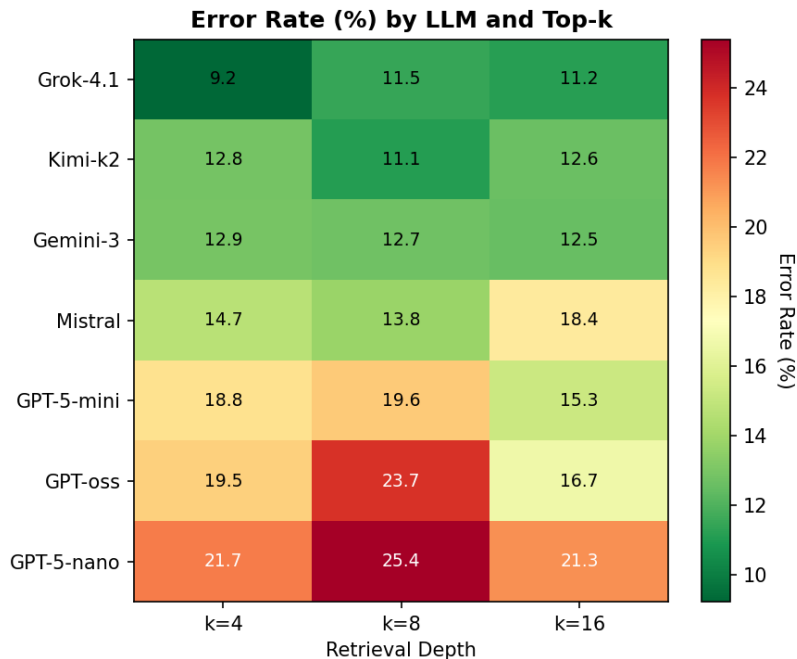


Figure 20: Error rates by LLM model and retrieval depth. Grok-4.1-fast achieves the lowest error rates across all configurations, while smaller models (GPT-5-nano) show consistently higher error rates.

Figure 21 breaks down errors into value errors (wrong answer) and reference errors (wrong citation) by model. Notably, value error rates vary more dramatically across models than reference errors, suggesting that answer extraction capability is more model-dependent than citation accuracy. Figure 22 shows how these error types vary with retrieval depth.

Several patterns emerge: (1) *Model capacity matters*: larger models (Grok-4.1-fast, Kimi-k2, Gemini-3-pro) achieve 9–13% error rates compared to 21–25% for GPT-5-nano. (2) *Retrieval depth trade-offs*: the relationship between top- k and error rate is non-monotonic; $k = 4$ and $k = 16$ often outperform $k = 8$. (3) *Error type differences*: value errors are more sensitive to model choice, while reference errors are more stable across configurations.

C.3 CONCRETE EXAMPLES AND ANALYSIS

To illustrate our classification criteria, we provide representative examples from each error category:

False Negative (Unnecessary Abstention).

Question: “What is the carbon footprint (kgCO₂eq) of training GPT-3?”

Ground truth: 552,000

Prediction: is.blank=true

Analysis: The model abstained despite the answer appearing in the retrieved context. This

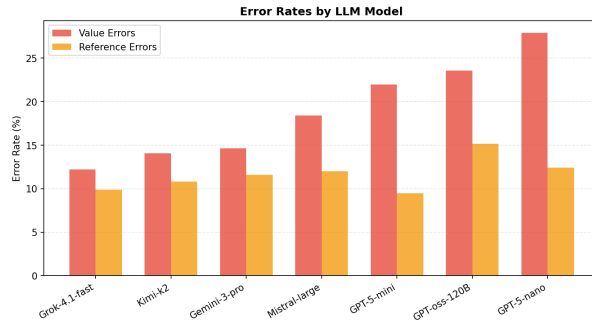


Figure 21: Value vs. reference error rates by LLM. Value errors show larger variation across models.

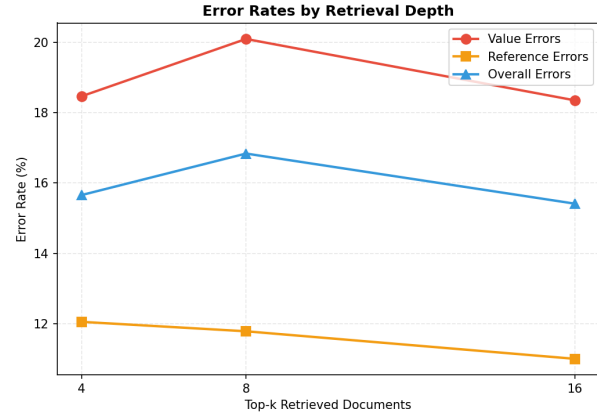


Figure 22: Error rates by retrieval depth. $k = 8$ shows highest value errors; reference errors decrease with k .

often occurs when the exact phrasing differs (“carbon emissions” vs “carbon footprint”) or when the model is uncertain about unit conversions.

Reference Mismatch.

Question: “What is the PUE of Google’s data centers?”

Ground truth value: 1.1, **refs:** [google2023]

Prediction value: 1.1, **refs:** [patterson2021]

Analysis: The answer is correct, but the model cited a survey paper that *mentions* Google’s PUE rather than Google’s official report. Both documents were in the retrieved context.

Value Selection Error.

Question: “What is the energy consumption (kWh) per inference for GPT-4?”

Ground truth: 0.0029

Prediction: 0.0017 ($\epsilon = 41.4\%$, $r = 0.59$)

Analysis: The retrieved context contained energy figures for multiple model sizes. The model selected the value for a smaller variant rather than the full GPT-4 model. Since $r = 0.59$ is not close to any power of 10, this is classified as value selection rather than unit conversion.

Rounding/Calculation Error.

Question: “What percentage of total US electricity consumption is attributed to data centers?”

Ground truth: 2.5

Prediction: 2.7 ($\epsilon = 8\%$)

Analysis: The model retrieved the correct source but performed a slightly incorrect calculation or read from an outdated figure in the same document. Since $\epsilon \leq 10\%$, this is classified as rounding/calculation error.

Unit Conversion Error.

Question: “How much water (liters) does training a large language model consume?”

Ground truth: 5,439,000

Prediction: 5,439 ($r = 0.001 = 10^{-3}$)

Analysis: The model confused “million liters” with “liters.” Since the ratio is within $\pm 5\%$ of 10^{-3} , this is classified as unit conversion error.

Type Mismatch.

Question: “What is the typical GPU utilization rate in data centers?”

Ground truth: 30 (numeric percentage)

Prediction: “30-50%” (range string)

Analysis: The ground truth expects a single numeric value, but the model output a range. This is classified as type mismatch regardless of whether the correct value falls within the range.

Discussion. Each dominant error category suggests different intervention strategies: **Unnecessary abstention** (26.8%) occurs when the model outputs `is_blank` despite sufficient context; the retry mechanism (Section 5.3) directly addresses this by expanding context when the model abstains. **Reference mismatch** (23.6%) indicates provenance tracking remains challenging; common causes include citing survey papers rather than original sources. **Value selection errors** (22.2%) arise when context contains multiple plausible values and the model selects the wrong one. **Rounding/calculation errors** (13.3%) are close misses indicating the system found correct information but made minor arithmetic mistakes. **Unit conversion errors** are notably rare (1.6%), indicating LLMs handle unit consistency well.

C.4 IMPLICATIONS FOR RAG SYSTEM DESIGN

Our error analysis suggests several directions for improving RAG system reliability:

1. **Reduce unnecessary abstention.** The retry mechanism with context expansion proves effective; further improvements could include confidence calibration or ensemble voting that ignores blank answers.
2. **Improve reference tracking.** Reference mismatch errors suggest LLMs struggle with precise attribution. Potential solutions include explicit reference extraction as a separate step or constraining answers to specific retrieved documents.
3. **Disambiguate similar values.** Value selection errors often occur when context contains multiple related values. Query planning with more specific sub-queries could help.
4. **Accept minor calculation errors.** Rounding errors represent fundamentally correct retrievals; evaluation metrics could distinguish between major and minor errors.
5. **Model selection matters.** The 16-point gap between Grok-4.1-fast (9.2%) and GPT-5-nano (25.4%) underscores that LLM capability is a dominant factor, potentially more impactful than retrieval optimizations.

D WATTBOT 2025 LEADERBOARD ANALYSIS

This appendix presents a comprehensive analysis of the WattBot 2025 competition leaderboard, examining the dynamics between public and private evaluation partitions. Understanding these dynamics is crucial for interpreting competition results and provides insights into method robustness, a key concern when deploying RAG systems in production environments where evaluation data may differ from development conditions.

D.1 FINAL LEADERBOARD

Figure 23 shows the official leaderboards of the WattBot 2025 Challenge. KohakuRAG (team “Kohaku-Lab”) achieved first place on both the public and private leaderboards, with scores of 0.902 and 0.861 respectively.

Public Private

This leaderboard is calculated with approximately 66% of the test data. The final results will be based on the other 34%, so the final standings may be different.

#	Team	Members	Score	Entries	Last	Solution
1	Kohaku-Lab		0.902	66	5d	
2	Attention Please		0.886	42	10d	

(a) Public leaderboard

Public Private

The private leaderboard is calculated with approximately 34% of the test data. This competition has completed. This leaderboard reflects the final standings.

#	Δ	Team	Members	Score	Entries	Last	Solution
1	—	Kohaku-Lab		0.901	66	5d	
2	+19	for84		0.858	1	22d	

(b) Private leaderboard (final)

Figure 23: WattBot 2025 Challenge leaderboards. (a) Public leaderboard visible during the competition. (b) Final private leaderboard revealed after the submission deadline. KohakuRAG maintained rank #1 across both partitions.

D.2 ANALYSIS METHODOLOGY

The WattBot 2025 competition employed a standard Kaggle-style evaluation protocol with two distinct test partitions: a *public* partition (66% of test questions) whose scores were visible during the competition, and a *private* partition (34%) revealed only after the submission deadline. Importantly, **both partitions are held-out test sets**: neither is directly accessible for training or validation. The key difference is that teams can observe public scores and iterate their submissions accordingly, while private scores remain completely hidden until the competition ends.

This creates an interesting evaluation dynamic: the public leaderboard measures performance on one test distribution that teams can *indirectly* optimize through submission feedback, while the private leaderboard measures transfer to a different test distribution with no optimization signal. A robust method should perform well on **both** partitions, as the final evaluation considers aggregate performance across the entire test set.

We analyze the complete leaderboard data to understand:

- How consistently do methods perform across the two test partitions?
- What patterns distinguish teams with stable vs. volatile performance?
- How does KohakuRAG’s performance compare across both evaluation dimensions?

D.3 SCORE TRANSITION ANALYSIS

The competition included 66 teams that submitted predictions evaluated on both partitions. Table 13 summarizes the key statistics describing score transitions from public to private evaluation.

Table 13: Summary statistics for public-to-private score transitions across all participating teams. The score delta (Δ) is computed as private score minus public score, where negative values indicate performance degradation on the private partition.

Metric	Value
Total teams	66
Mean Δ (private – public)	-0.0106
Standard deviation of Δ	0.0452
Median Δ	-0.0160
Minimum Δ	-0.2130
Maximum Δ	0.0980
Teams with score decrease	43 (65.2%)
Teams with score increase	22 (33.3%)
Pearson correlation (public score, Δ)	-0.389

Several observations emerge from these statistics. First, the majority of teams (65%) experienced score degradation on the private partition, with a mean delta of -0.0106 . This systematic decrease is expected: teams naturally gravitate toward configurations that score well on the observable public partition, which may not transfer perfectly to the private partition’s question distribution.

Second, we observe a moderate negative correlation ($r = -0.389$) between public scores and score deltas. This indicates that teams with higher public scores tended to experience larger drops on the private partition.

However, this does *not* imply that achieving high public scores is undesirable; it simply reflects that there is variance between the two test distributions, and methods optimized (even indirectly) for one may not perfectly transfer to the other. Figure 24 visualizes this relationship.

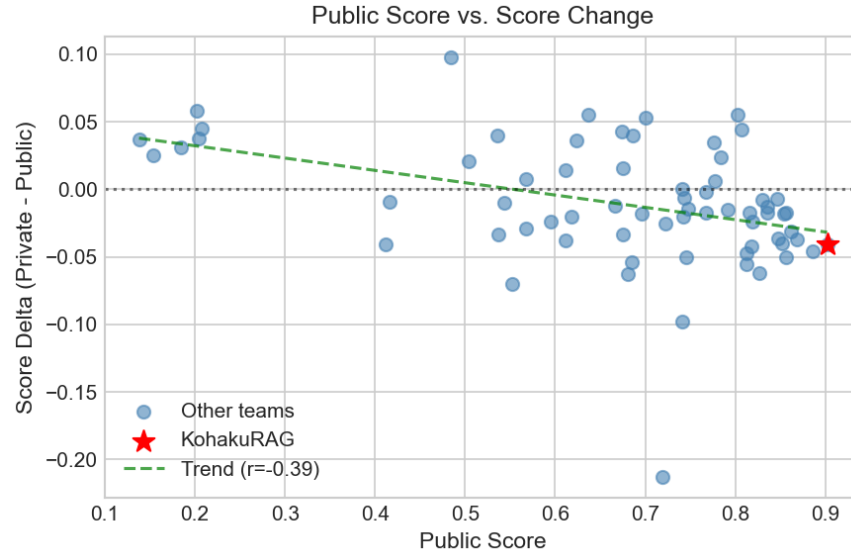


Figure 24: Relationship between public score and score delta. Each point represents a team; KohakuRAG is marked with a red star. The negative trend reflects variance between the two test distributions rather than indicating that high public scores are inherently problematic.

Figure 25 shows the distribution of score deltas across all teams, providing context for evaluating any individual team's performance transition. The distribution is left-skewed, with more teams experiencing score drops than gains.

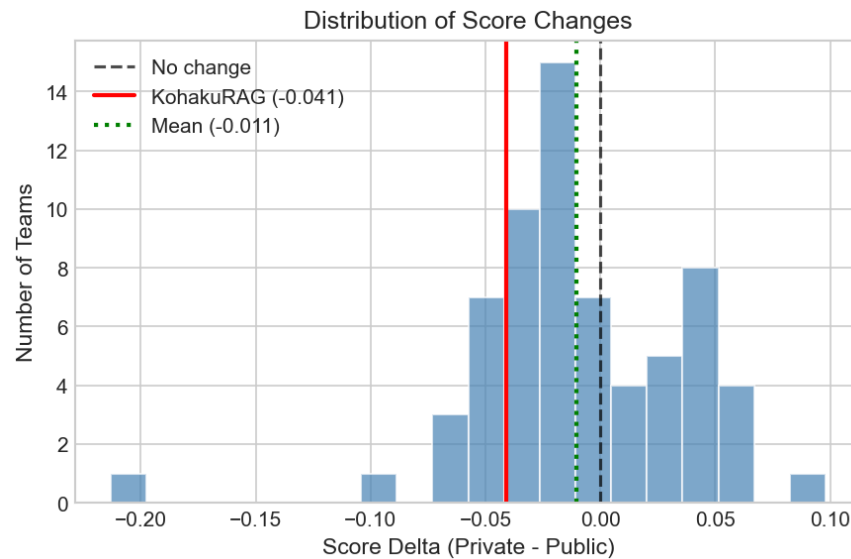


Figure 25: Distribution of score deltas (private – public) across all teams. KohakuRAG's delta is marked for reference.

D.4 RANK MOVEMENT PATTERNS

Top-10 Teams. To better understand the dynamics among competitive submissions, we examine the top 10 teams from the public leaderboard and their corresponding private performance. Table 14 presents these transitions in detail.

Table 14: Public-to-private transition for the top 10 public leaderboard teams. The Δ column shows score change, and Rank Δ shows position change (positive = improved). KohakuRAG ($\star$) was the only team to maintain first place across both partitions.

Pub Rank	Pub Score	Priv Score	Priv Rank	Score Δ	Rank Δ
#1 $\star$	0.902	0.861	#1	-0.041	0
#2	0.886	0.840	#4	-0.046	-2
#3	0.868	0.831	#9	-0.037	-6
#4	0.862	0.831	#8	-0.031	-4
#5	0.856	0.806	#17	-0.050	-12
#6	0.856	0.839	#6	-0.017	0
#7	0.854	0.836	#7	-0.018	0
#8	0.852	0.812	#13	-0.040	-5
#9	0.848	0.812	#14	-0.036	-5
#10	0.846	0.839	#5	-0.007	+5

The top 10 public teams collectively show a mean delta of -0.0323 , larger in magnitude than the overall mean of -0.0106 . This is consistent with the correlation analysis: teams at the performance frontier tend to show more variance between partitions. Importantly, despite these deltas, the top public teams generally remained competitive on the private leaderboard as well; the correlation reflects *relative* shifts, not absolute failure.

Figure 26 visualizes rank transitions for the top 20 public teams, illustrating the substantial reshuffling that occurred when private scores were revealed.

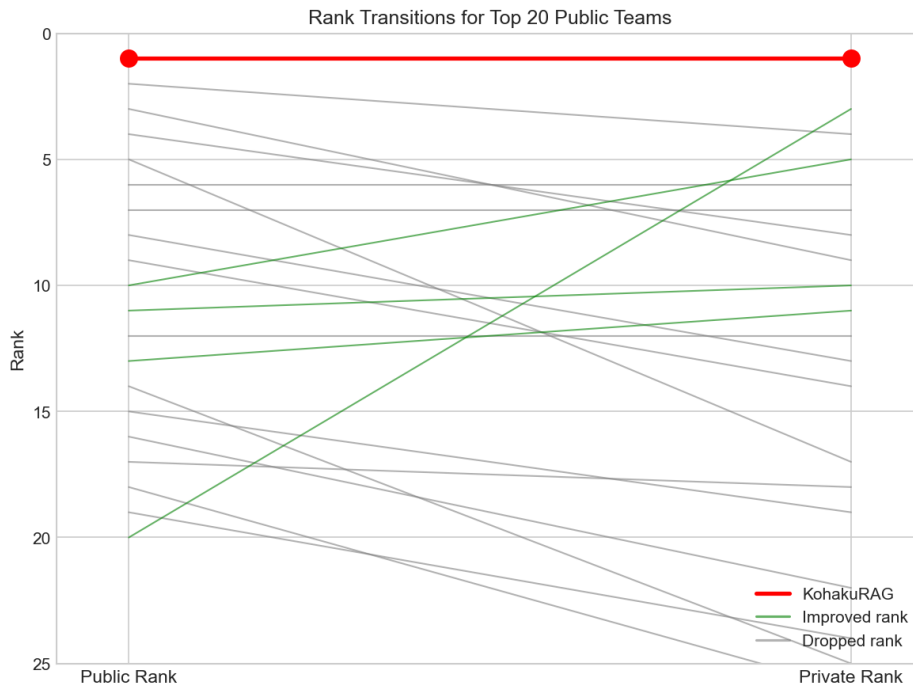


Figure 26: Rank transitions from public to private leaderboard for top 20 public teams. Lines connect each team’s public rank (left) to private rank (right). KohakuRAG (red) maintained rank #1; green lines indicate teams that improved, gray lines indicate teams that dropped.

Stability by Rank Band. To determine whether the pattern is concentrated among top performers or distributed throughout the leaderboard, we partition teams into rank bands (Table 15).

Table 15: Score delta statistics stratified by public rank band. Top-ranked teams show more negative mean deltas, while mid-to-low ranked teams show near-zero or positive mean deltas.

Rank Band	n	Mean Δ	Std Δ	Median Δ
1–10	10	-0.0323	0.0132	-0.0365
11–20	10	-0.0241	0.0289	-0.0205
21–30	10	+0.0016	0.0284	-0.0040
31–50	20	-0.0181	0.0597	-0.0190
51–100	16	+0.0131	0.0414	+0.0230

Teams in the top rank bands (1–10 and 11–20) exhibit more negative mean deltas compared to lower-ranked teams. The 21–30 and 51–100 bands show near-zero or slightly positive mean deltas. This pattern has a natural explanation: lower-ranked teams had more “room to improve” on the private partition (regression to the mean), while top teams were already near the performance ceiling and thus more likely to show decreases. This does *not* suggest that targeting mid-tier public rankings leads to better outcomes, since teams that jumped in rank still achieved lower final scores than consistently top-performing teams like KohakuRAG.

Largest Rank Changes. Table 16 highlights the most dramatic rank changes in both directions.

Table 16: Teams with the largest rank improvements (left) and largest rank drops (right). Notable gainers came from outside the top 20 public, while notable losers fell from top-10 positions.

Public	Private	Δ Rank	Δ Score	Public	Private	Δ Rank	Δ Score
#21	#2	+19	+0.055	#5	#17	-12	-0.050
#20	#3	+17	+0.044	#14	#25	-11	-0.062
#25	#15	+10	+0.035	#18	#26	-8	-0.055
#23	#16	+7	+0.024	#3	#9	-6	-0.037
#10	#5	+5	-0.007	#16	#22	-6	-0.042

The most dramatic rank improvements came from teams initially ranked #20–25, with the largest gainer jumping 19 positions from #21 to #2. However, it is important to note that these teams’ *final private scores* (0.858, 0.851) were still lower than KohakuRAG’s (0.861). The rank improvement reflects relative movement, not absolute superiority; their methods happened to transfer better to the private partition’s specific question distribution, but they did not achieve the highest overall performance.

Conversely, some top-10 public teams experienced significant drops, with one team falling 12 positions. These cases illustrate the variance between test partitions, but do not indicate that high public performance is undesirable. The goal remains to maximize performance on **both** partitions, which KohakuRAG achieved by maintaining #1 ranking across both.

D.5 KOHAKURAG PERFORMANCE AND IMPLICATIONS

Our Results. KohakuRAG achieved a public score of 0.902 (rank #1) and a private score of 0.861 (rank #1), with a delta of -0.041 . The key result is that **KohakuRAG achieved the highest score on both partitions independently**, maintaining first place across both test distributions. This is the ideal outcome: strong performance that transfers consistently.

While our delta magnitude is larger than 68% of teams, this metric is less meaningful than absolute performance. Many teams with small deltas simply had mediocre scores on both partitions. What matters is achieving high scores on *both* test sets, which KohakuRAG accomplished.

Our ensemble-based approach contributed to this consistency. By aggregating predictions from multiple model runs with abstention-aware voting, we reduced variance in individual predictions. The ensemble does not guarantee zero delta, as the two test partitions genuinely differ in their question distributions, but it helps ensure that performance remains competitive across both evaluation conditions rather than being optimized for one at the expense of the other.

Broader Implications. The patterns observed have implications for RAG system development:

1. **Both test partitions matter equally.** Teams that “improved” from public to private often did so from a lower baseline and still achieved lower final scores than consistently top-performing teams.
2. **Rank changes reflect distribution variance, not method quality.** The teams that jumped from #20→#2 did not develop superior approaches; they happened to have methods that transferred better to the private partition’s specific characteristics.
3. **Ensemble methods help maintain consistency.** KohakuRAG’s ensemble approach contributed to achieving #1 on both partitions by reducing variance across evaluation conditions.
4. **Absolute performance trumps relative stability.** Having a small delta is meaningless if both scores are low. The objective is maximizing aggregate performance on both partitions.

E LIMITATIONS

Our work has several limitations worth noting. Our document parsing pipeline relies on rule-based heuristics for section detection, paragraph segmentation, and sentence boundary identification. While effective for well-structured technical documents, these heuristics may fail on irregular layouts, multi-column formats, or documents with non-standard visual hierarchies, propagating errors to downstream retrieval.

Reference attribution remains challenging. Our error analysis reveals that 23.6% of errors involve citation mismatch—the LLM produces correct answers but cites survey papers or secondary sources rather than original documents. This occurs because multiple documents in the corpus discuss overlapping topics, and the model lacks explicit mechanisms to prefer primary sources over derivative ones.

Ensemble inference, while improving robustness, incurs significant computational overhead. Our best configurations require 7–15 independent runs, linearly scaling both latency and API costs. For latency-sensitive or cost-constrained deployments, this overhead may be prohibitive despite the accuracy gains.

Our multimodal integration is relatively shallow. Image captions are generated independently without document context, and direct image retrieval yielded only marginal improvements (+1.2%) over caption-based approaches. Complex visualizations such as multi-panel figures or data-dense charts remain challenging to retrieve and interpret effectively.

Finally, our evaluation is confined to a single domain—AI energy consumption across 32 documents. While we achieve strong results on WattBot 2025, generalization to other domains, larger corpora, or different question types remains unvalidated.

F FUTURE WORK

Several directions merit further investigation. Replacing rule-based parsing with learned approaches such as LayoutLM or DocFormer could improve robustness to diverse document layouts and reduce error propagation from incorrect structure extraction.

Addressing reference mismatch may require reference-constrained decoding that restricts citations to top-ranked passages, or explicit attribution verification as a post-processing step. Training with citation supervision could also improve the model’s ability to identify primary sources.

The computational cost of ensemble inference motivates research into efficient alternatives. Confidence-based selective ensembling could apply majority voting only for uncertain predictions, while knowledge distillation could transfer ensemble robustness into a single model. Adaptive retrieval that predicts when additional context is needed—rather than using fixed k —could further reduce unnecessary computation.

Deeper multimodal integration through cross-modal attention and structured extraction of charts and tables could better capture visual information in technical documents. Finally, validation on diverse benchmarks spanning multiple domains, scales, and question types would establish broader applicability of our approach.